

DOCUMENTATION

CVE Report Generator

Complete Guide to `cve_report.py`

A beginner-to-advanced guide to the Linux-kernel CVE report generator: what it does, the concepts behind it, how it is built, which sources it trusts and how reliable they are.

- Concepts explained from first principles
- Architecture, dependencies and data sources
- Function-by-function code walkthrough
- Source confirmation and reliability assessment
- 22 diagrams and flowcharts

Written against commit 7e38fe1 · 6 October 2026

- **Documents:** `cve_report.py` — a single-file Python 3 command-line tool (standard library only)
- **Companions:** `find_module_new.py` · `configs/` · `README.md` · `.claude/commands/cve.md`
- **Written against:** commit `7e38fe1` (2026-10-06). Function links in Appendix B point at line numbers of that version.

How to read this guide. It is organised from beginner to advanced. You do not have to read it in order — pick the row in the table below that matches you.

You are...	Start at	You will learn
Brand new, just want a report	Part 1	What the tool is, how to run it, how to read its output
New to security jargon	Part 2	CVE, CVSS, CWE, EPSS, KEV, "dyad", "backport", <code>CONFIG_...</code> in plain words
Need to operate or install it	Part 3	Architecture, dependencies, every data source, what happens when something is missing
Need to understand or modify the code	Part 4 and Part 5	Function-by-function walkthrough, design decisions, extension recipes
Need to decide how far to trust it	Part 6	Which sources are official, how each can fail, how to verify any claim yourself

Contents

- [Part 1 — The 5-minute overview \(Beginner\)](#)
- [Part 2 — Concepts you need \(Beginner → Intermediate\)](#)
- [Part 3 — Architecture and dependencies \(Intermediate\)](#)
- [Part 4 — Code walkthrough \(Intermediate\)](#)
- [Part 5 — Advanced topics](#)
- [Part 6 — Official sources and reliability](#)
- [Part 7 — Advantages, limits and when to use what](#)
- [Appendix A — Command-line reference](#)
- [Appendix B — Function index](#)
- [Appendix C — A real run, annotated](#)
- [Appendix D — FAQ](#)

Part 1 — The 5-minute overview (Beginner)

1.1 What is this tool?

`cve_report.py` takes **one vulnerability ID** (for example `CVE-2026-53359`) and produces **one report** about it.

To build that report it:

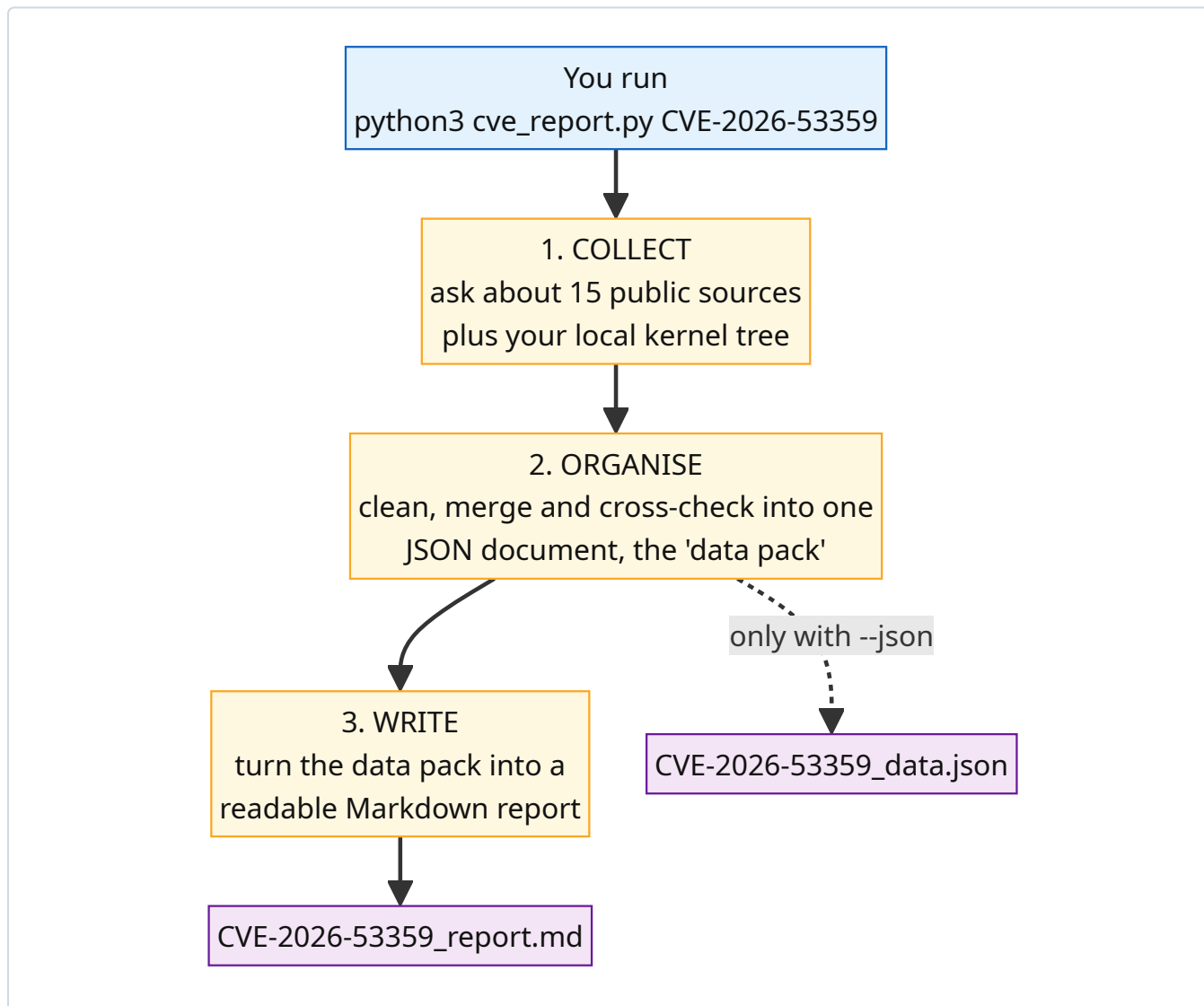
1. asks about 15 public websites and APIs what they know about the vulnerability,
2. works out **which Linux kernel versions are broken and which releases contain the fix**,
3. works out **which kernel module** (if any) contains the broken code, and — using your own kernel configuration files — whether that module is built for your kernels,
4. optionally asks an internal script **how many of your servers actually load that module**, and
5. writes everything up as a Markdown document.

It is built for **Linux-kernel CVEs** (the ones whose "assigner" is kernel.org). Other CVEs still produce a report, but without the kernel-specific parts.

1.2 The problem it solves

Question an analyst must answer	Doing it by hand	With the tool
What is the bug and how severe is it?	Open CVE.org, then NVD, compare scores	One table: "At a glance"
Is anyone exploiting it?	Check CISA's KEV list, FIRST EPSS, Exploit-DB	Same table
Which kernel versions are affected and which release fixes each branch?	Read git history, match commits to releases by hand	Per-branch fix table, built from the kernel team's own data
Which <code>.ko</code> module is it in, and is it compiled for our kernels?	Read Makefiles and Kconfig, check <code>.config</code>	Per-branch module table
Do any of our servers load that module?	Query the fleet inventory	"Fleet exposure" table
What do Debian / TuxCare / the press say?	Many browser tabs	Sections 8 and 9
Where did each fact come from?	You remember (or not)	"Provenance" section + a console list of what was skipped and why

1.3 Inputs and outputs



The **data pack** is the heart of the tool: a single JSON object holding every fact the tool found. The report is just a human-friendly rendering of it. (Its full structure is in [section 5.2.](#))

1.4 Your first run

Requirements for the simplest run: Python 3.7 or newer (the repo has been run with 3.12) and internet access. Nothing needs to be installed — the script uses only Python's standard library.

```
cd CVE_Report

# Simplest run: no LLM, no internal tools, just public data (took ~27 s in a test)
python3 cve_report.py CVE-2026-53359 --template --no-module-resolve --no-module-stats

# Full run: LLM-written report + per-branch module table + fleet exposure
zllm start                                # once, in another terminal
python3 cve_report.py CVE-2026-53359
```

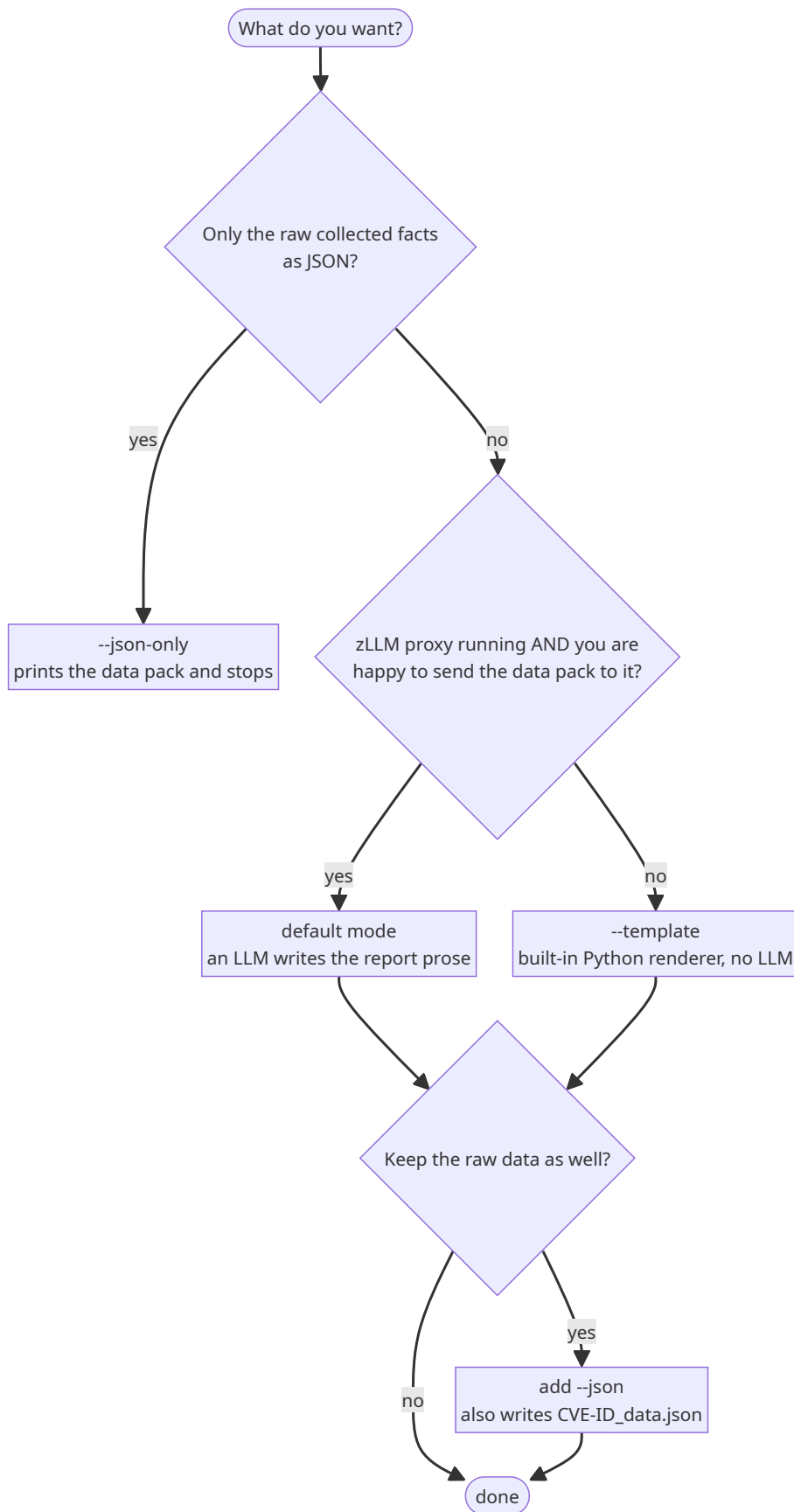
Both write `CVE-2026-53359_report.md` in the current directory.

While it runs you will see progress lines on **stderr** (what each means is explained in [Appendix C](#)):

```
[*] CVE-2026-53359: fetching cvelistV5 record ...
[*] fetching dyad / NVD / EPSS / KEV ...
[*] fetching Exploit-DB ...
...
- not included: Per-branch module resolution (disabled (--no-module-resolve))
- not included: Fleet module-usage stats (disabled (--no-module-stats))
[+] report written: CVE-2026-53359_report.md
```

The lines starting with `- not included:` are deliberate: the tool tells you **which sources it did not use and why**, so a missing section is never a mystery.

1.5 Choosing a mode



Mode	Command	Needs the LLM proxy?	Output style
Default	<code>python3 cve_report.py CVE-...</code>	Yes — exits with an error if it is not reachable	Fluent prose written by the LLM from the data pack
Template	add <code>--template</code>	No	Fixed, fully predictable Markdown layout
Data only	add <code>--json-only</code>	No	The raw JSON data pack on stdout
Both	add <code>--json</code>	Depends on mode	Report and <code>CVE-..._data.json</code>
Preview	add <code>--stdout</code>	Depends on mode	Prints the report instead of writing a file

"Template" is not "offline". `--template` only means *no LLM*. Collecting the data still needs the internet.

1.6 What is inside a report

#	Section	The question it answers	Mainly built from
1	At a glance	How bad, how likely to be exploited, is it being exploited?	CVE record, NVD, EPSS, CISA KEV/SSVC, Exploit-DB
2	Affected component	Which subsystem / file / function / module / <code>CONFIG_</code> option?	CVE record, commit diff, Makefiles, local tree + <code>.config</code> files
3	Affected & fixed versions	Since when is it broken? Which release fixes each branch? Which branches will never get a fix?	kernel.org dyad file, commit dates
3b	Fleet exposure	Is the module built-in, not loaded anywhere, or loaded on N servers per region?	<code>find_module_new.py</code> , <code>module_stats.py</code>
4	Vulnerability details	What is the weakness? Upstream description; TuxCare status	CVE record, CWE, TuxCare
5	The fix	Which commit, by whom, when, how big, backports per branch?	git (local or git.kernel.org)
6	Mitigations & detection	What should I do?	Derived from the above
7	References	Where are the primary documents?	CVE record, NVD, discussion lists
8	Debian	Is Debian fixed, per suite?	Debian Security Tracker
9	Media & community coverage	Has anyone written about it?	Feedly, Hacker Wire, news sites
10	Provenance	Which sources were used, when was it generated?	The tool itself

The Red Hat, Ubuntu, SUSE, Arch and OSV sections are **deliberately not rendered** (see [section 5.8](#)).

1.7 The ten flags you will actually use

Flag	What it does
<code>-o FILE</code>	Write the report to <code>FILE</code> instead of <code><CVE>_report.md</code>
<code>--stdout</code>	Print the report; write no report file

Flag	What it does
<code>--template</code>	Skip the LLM, use the built-in renderer
<code>--json</code> / <code>--json-only</code>	Also save the data pack / print only the data pack
<code>--no-diff</code>	Do not fetch commit patches (faster, loses dates, authors, functions)
<code>--no-vendor</code>	Skip Exploit-DB, Debian, linuxkernelcves, TuxCare
<code>--no-media</code>	Skip Feedly, Hacker Wire and the three news sites
<code>--no-module-resolve</code>	Skip the per-branch module table (the slowest optional step)
<code>--no-module-stats</code>	Skip the fleet-exposure lookup
<code>--linux-src PATH</code> / <code>--configs-dir PATH</code>	Point at your Linux clone / your <code>config-*</code> files

The complete list is in [Appendix A](#).

Part 2 — Concepts you need (Beginner → Intermediate)

2.1 Vulnerability vocabulary

Term	Plain-language meaning	Where it shows up in this tool
CVE	A unique ID for one publicly known vulnerability, format <code>CVE-YEAR-NUMBER</code> .	The tool's only input. Validated by the regex <code>^CVE-\d{4}-\d{4,}\\$</code> .
CNA (CVE Numbering Authority)	An organisation allowed to assign CVE IDs and write the official record. kernel.org is a CNA and writes almost all Linux-kernel CVEs.	<code>assignerShortName == "linux"</code> is how the tool decides a CVE is a <i>kernel CVE</i> (<code>is_kernel</code>).
CVE record	The official JSON document describing a CVE. Contains a <code>cna</code> block (written by the CNA) and optional <code>adp</code> blocks (added by other authorised publishers).	Fetches first; everything else builds on it.
ADP (Authorized Data Publisher)	A third party allowed to <i>add</i> data to someone else's record. CISA's ADP adds severity, CWE and SSVC.	<code>parse_cvss</code> , <code>parse_cwe</code> , <code>parse_ssvc</code> read <code>containers.adp[]</code> .
cvelistV5	The public GitHub repository holding every CVE record as a JSON file.	Primary source (raw file URL).
CVSS	A 0–10 score of <i>technical</i> severity, plus a vector string that records how it was computed.	"CVSS" row. See decoder below.
CWE	A catalogue of <i>kinds</i> of software weakness, e.g. CWE-416 = Use After Free.	"Weakness" row.
EPSS	A probability (0–1) that a CVE will be exploited in the wild in the next 30 days, plus a percentile rank among all CVEs. Published by FIRST.	"EPSS" row.
KEV	CISA's <i>Known Exploited Vulnerabilities</i> catalogue: CVEs with evidence of real exploitation .	"CISA KEV" row.
SSVC	CISA's decision-tree labels: <i>Exploitation</i> (<code>none</code> / <code>poc</code> / <code>active</code>), <i>Automatable</i> (<code>yes</code> / <code>no</code>), <i>Technical impact</i> (<code>partial</code> / <code>total</code>).	"CISA SSVC" row.
NVD	The US National Vulnerability Database (NIST). Takes CVE records and adds scoring and analysis.	"NVD status" row; extra CVSS/CWE/CPE/references.
CPE	A machine-readable name for a product and version range.	"Vulnerable version ranges (CPE)".
PoC / exploit	Code that demonstrates / abuses the bug.	Exploit-DB lookup.

Severity bands (what `_sev_from_score` implements when a source gives only a number):

Score	Label
0.0	NONE
0.1 – 3.9	LOW
4.0 – 6.9	MEDIUM
7.0 – 8.9	HIGH
9.0 – 10.0	CRITICAL

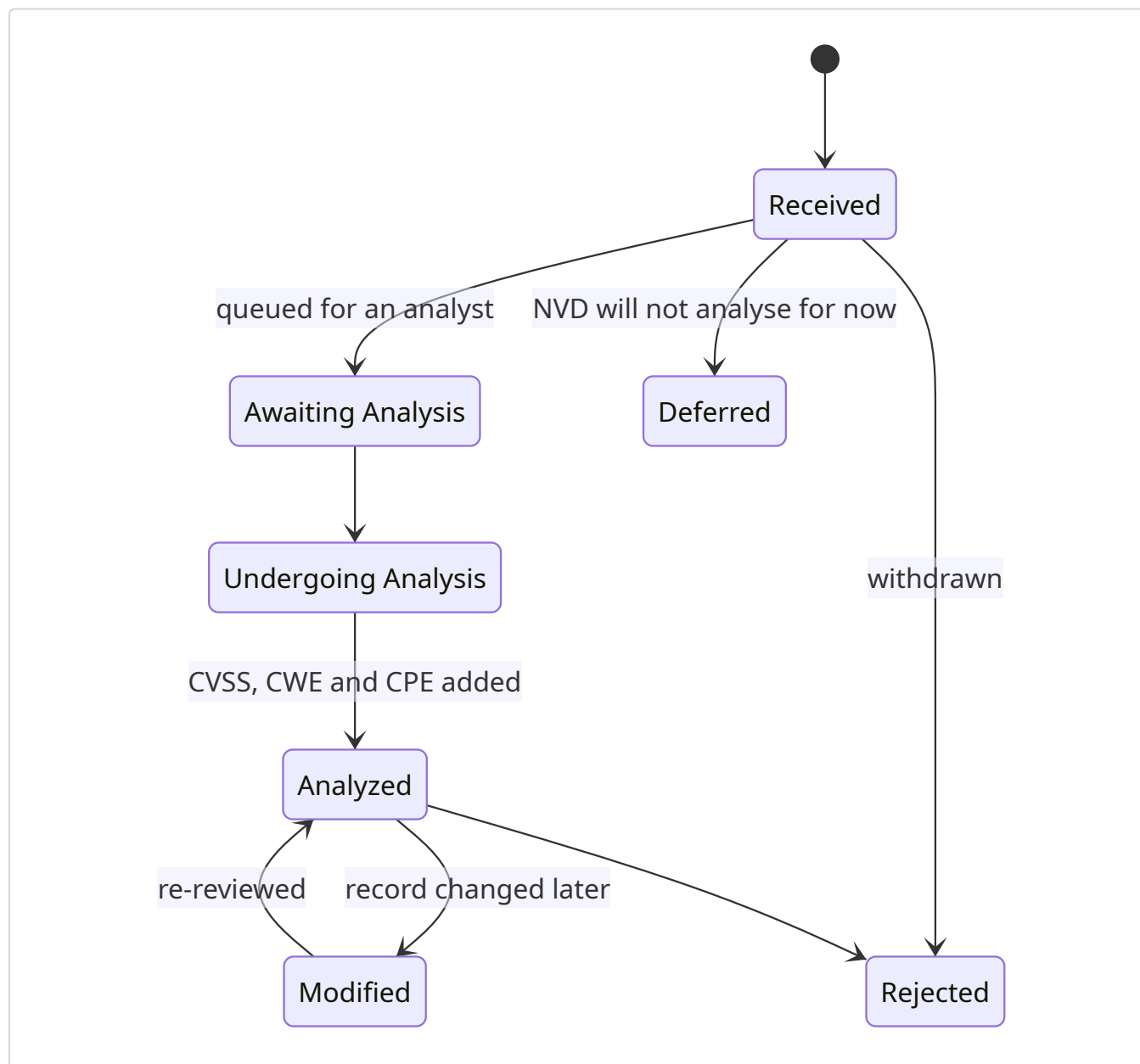
Decoding a CVSS vector — the real one from the sample run,

CVSS:3.1/AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H → **8.8 HIGH**:

Part	Meaning
AV:L	Attack Vector: Local (attacker needs local access)
AC:L	Attack Complexity: Low
PR:L	Privileges Required: Low (an ordinary account)
UI:N	User Interaction: None
S:C	Scope: Changed (can affect beyond the vulnerable component, e.g. guest → host)
C:H I:H A:H	Confidentiality / Integrity / Availability impact: all High

CVSS is severity, not risk. A high CVSS on a module you never load is low risk for you. That is exactly why this tool also reports EPSS, KEV and *fleet exposure*.

NVD processing states (the `vulnStatus` field the tool prints as "NVD status"; simplified):



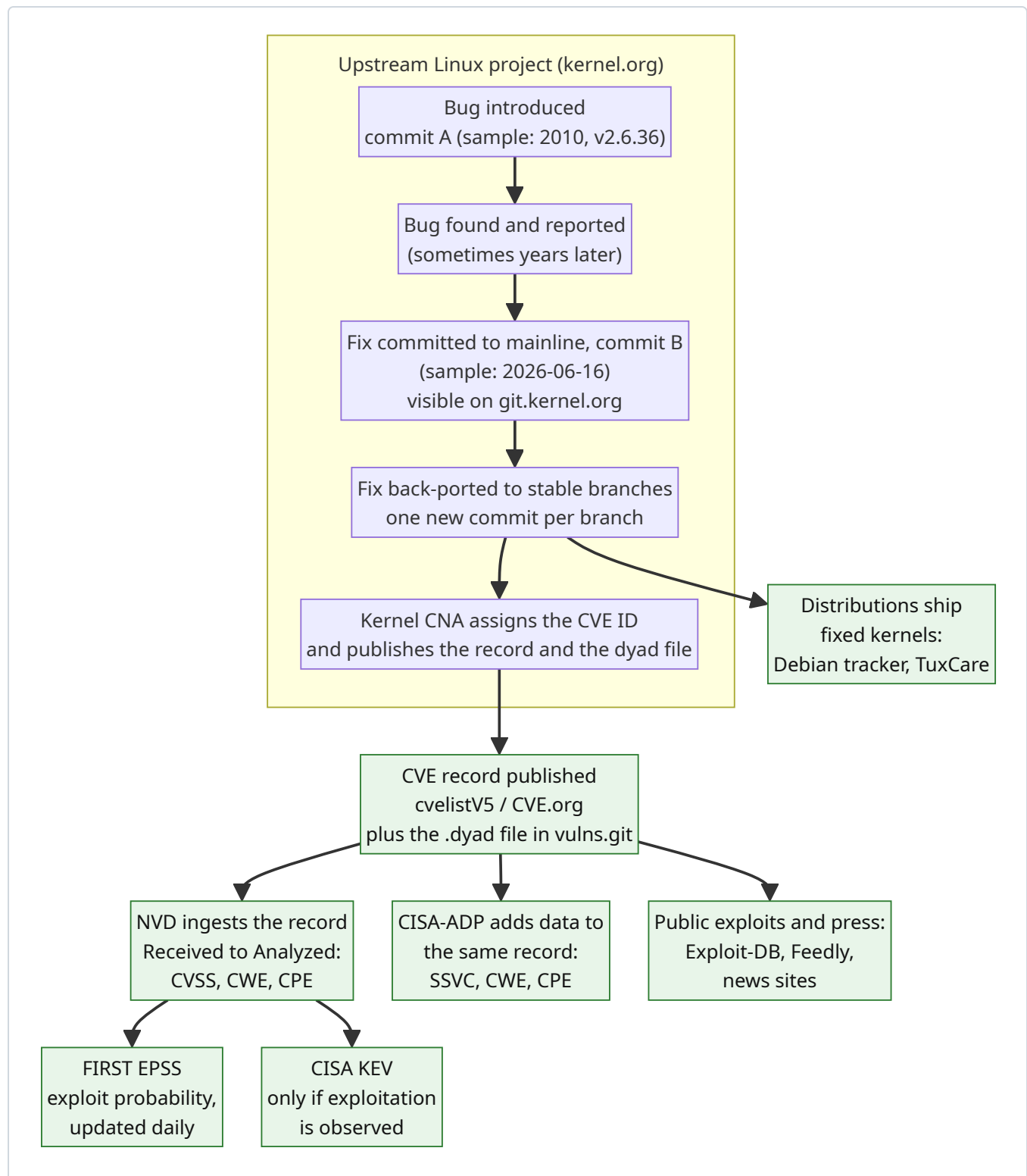
A CVE in state `Received` (like `CVE-2026-80844` in the repo's sample data) has **no NVD score and no NVD CWE yet**. That is normal, not an error — the report simply shows what the CNA supplied.

2.2 Linux-kernel vocabulary

Term	Plain-language meaning
Mainline	Linus Torvalds's main development tree. New features and the <i>first</i> version of every fix land here.
Stable branch (<code>6.12.y</code>)	A long-lived branch that receives only bug fixes. Releases on it look like <code>6.12.95</code> . The tool derives the branch from a release with <code>_branch_of("6.12.95") → "6.12.y"</code> .
Commit	One recorded change, identified by a 40-hex-digit hash (often shown as the first 12).
Backport	Copying a mainline fix onto an older stable branch. Each backport is a <i>different commit</i> with a <i>different hash</i> .

Term	Plain-language meaning
Introduced / fixed	The commit that created the bug vs. the commit that repaired it.
EOL branch	A branch that is no longer maintained. If it was vulnerable, it will never be fixed. The tool lists these as "affected but not fixed".
Dyad	A pair (vulnerable commit/release, fixed commit/release). The kernel CNA publishes one .dyad file per CVE listing all pairs.
Subsystem	A major area of the kernel (KVM, netfilter, ext4...). The commit title usually starts with it: KVM: x86: Fix
Kernel module (.ko)	Code compiled as a separately loadable file. Present on a running system only if loaded (lsmod).
Built-in (vmlinux)	Code compiled into the kernel image itself. Always present; there is nothing to "not load".
Kconfig / .config / CONFIG_F00	Build-time switches. CONFIG_F00=y → built-in, =m → module, <i>not set</i> → not compiled at all.
Makefile obj-\$(CONFIG_F00) += foo.o	The line that ties a source file to a CONFIG_ switch. The tool reads these to answer "which module is this .c file part of?"
git worktree	An extra checkout of the same repository in another folder, sharing history. The tool makes throw-away ones to read each branch's Makefiles without disturbing your clone.
Diff / hunk	A diff is a list of changes. Each <i>hunk</i> starts with a header like @@ -120,7 +120,9 @@ some_function(...) .
Diffstat	Summary of a patch: "+6 / -4 across 1 file".

2.3 The life of a kernel vulnerability — and where each source learns about it



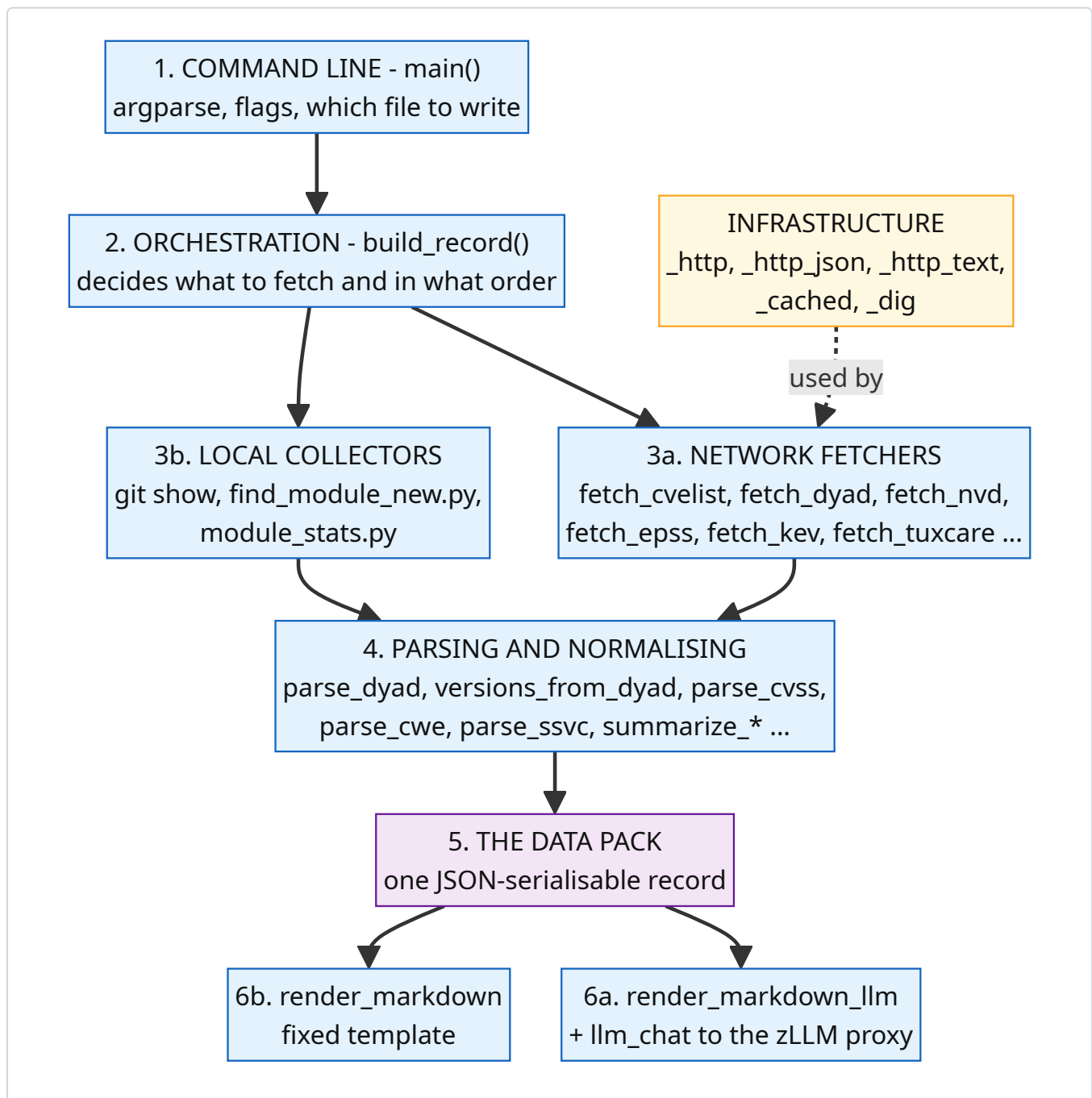
Green boxes are the places `cve_report.py` reads. The point of the diagram: **no single place has the whole story**. The CVE record has the text but not the per-branch releases; the dyad file has the releases but not the score; NVD has the score later; EPSS and KEV are separate feeds; distros have their own status. The tool's job is to join them.

2.4 Programming concepts used by the tool

Concept	Meaning	Where it appears
REST API / JSON	Ask a URL, get structured data back.	NVD, EPSS, cveawg, OSV, Red Hat, Arch, WordPress <code>/wp-json/</code> , Blogger feeds
Bulk dump	No per-item API; download one big file and search it locally.	CISA KEV, Debian tracker, linuxkernelcves.com
Scraping	Pulling data out of a web page meant for humans. Fragile.	TuxCare (regex over HTML)
Embedded structured data	A page that carries a machine-readable JSON block inside its HTML. More robust than prose scraping.	Feedly (<code>__NEXT_DATA__</code>), Hacker Wire (<code>ld+json</code>)
HTTP status codes	<code>200</code> ok · <code>404</code> "no such thing" (often normal here) · <code>403/429/503</code> "slow down / try later".	<code>_http</code> retry logic
Retry with back-off	Try again after a growing pause.	<code>_http</code>
TTL cache	Keep a downloaded file for <i>N</i> hours before re-downloading.	<code>_cached</code>
Fallback chain	Try source A; if unusable, source B.	<code>fetch_cvelist</code> , <code>parse_cwe</code> , <code>fetch_patch</code>
Graceful degradation	If an optional piece is missing, produce the report without it.	Almost everything except the CVE record and the LLM
Regular expression (regex)	A text-matching pattern.	Parsing dyad files, patches, Makefiles, HTML
Subprocess	Running another program (<code>git</code> , <code>module_stats.py</code>).	<code>_fetch_patch_local</code> , <code>fetch_module_stats</code>
LLM / OpenAI-compatible API	A language model reached through <code>POST /chat/completions</code> with a <i>system prompt</i> (rules) and a <i>user message</i> (task + data).	<code>llm_chat</code> , <code>render_markdown_llm</code>
Grounding	Forcing an LLM to use only supplied facts.	The system prompt: "Use ONLY the facts present in that JSON"
Temperature	Randomness of LLM output (0 = most deterministic). The tool uses <code>0.2</code> .	<code>llm_chat</code>

Part 3 — Architecture and dependencies (Intermediate)

3.1 Architecture at a glance



Design in one sentence: **collect** → **normalise** → **one JSON record** → **render**. Because rendering only ever sees the JSON record, you can swap or add renderers without touching any fetcher.

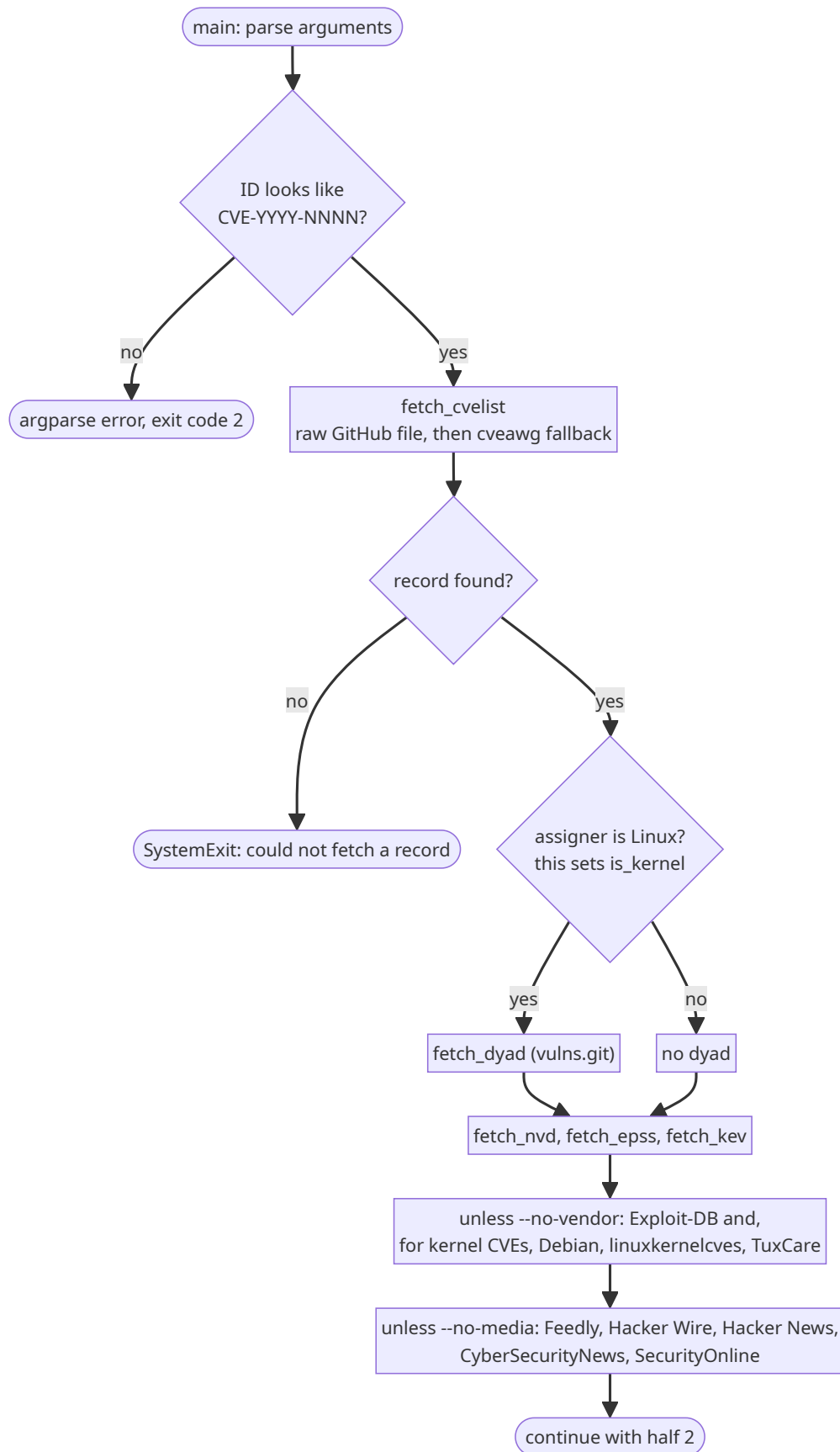
3.2 How the file is organised

The source file is divided by banner comments. Reading it top to bottom follows the same order as a run.

Banner in the file	Contents	Role
Module docstring	Usage, source list, env vars	Human documentation
Constants	URLs, cache path, defaults, the LLM prompts, <code>CWE_KEYWORDS</code>	Configuration in code
HTTP helpers	<code>_http</code> , <code>_http_json</code> , <code>_http_text</code> , <code>llm_chat</code> , <code>_cached</code>	Network + cache plumbing
URL helpers	<code>_cve_parts</code> , <code>cvelist_url</code> , <code>cvelist_blob_url</code>	Build the CVE-list URL
Source fetchers	<code>fetch_*</code> , patch parsing, Makefile/module helpers	One function per source
Small utils	<code>_dig</code> , <code>_sev_from_score</code> , <code>_branch_of</code> , <code>classify_cwe</code>	Helpers
Parsing	<code>parse_*</code> , <code>versions_*</code> , <code>summarize_*</code> , <code>categorize_refs</code> , ...	Raw → normalised
Build record	<code>build_record</code>	The orchestrator
Rendering	<code>_slim_*</code> , <code>render_markdown_llm</code> , <code>render_markdown</code>	Record → Markdown
Main	<code>main</code>	CLI entry point

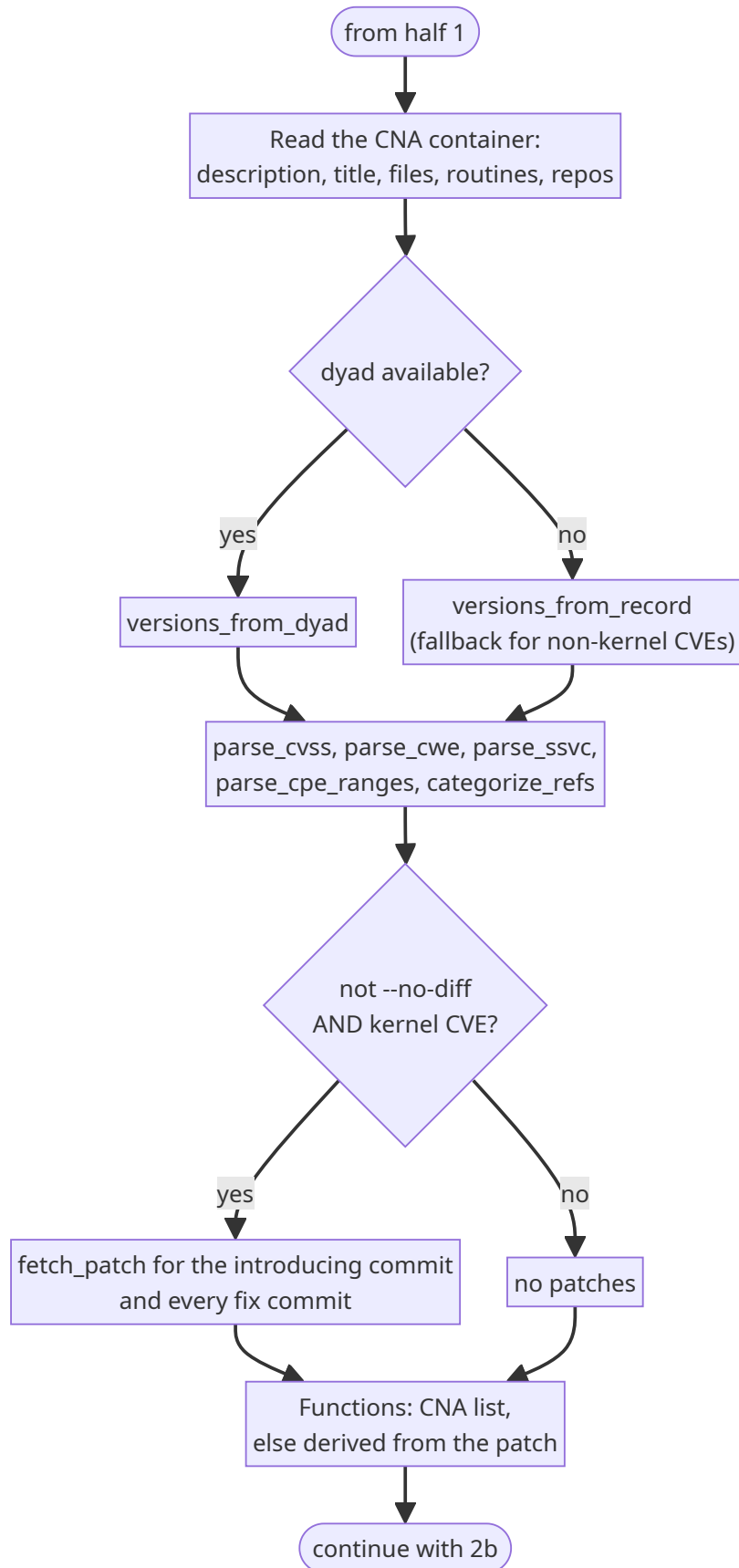
3.3 The pipeline in two halves

Half 1 — collecting

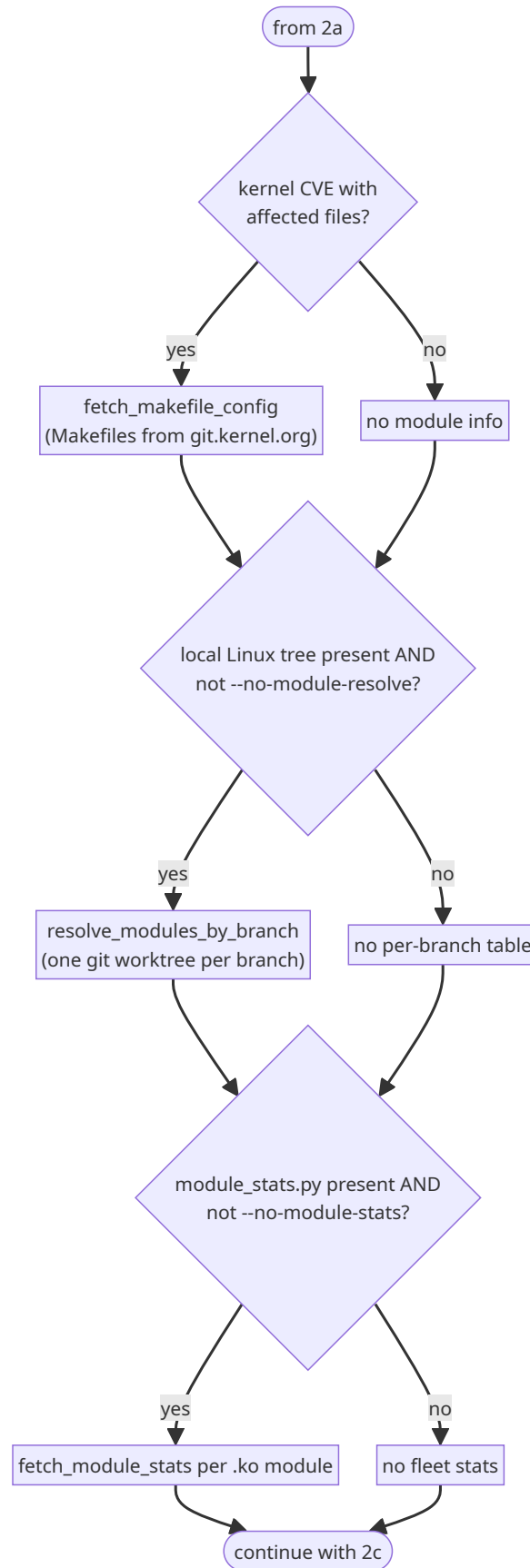


Half 2 — enriching and writing (three parts)

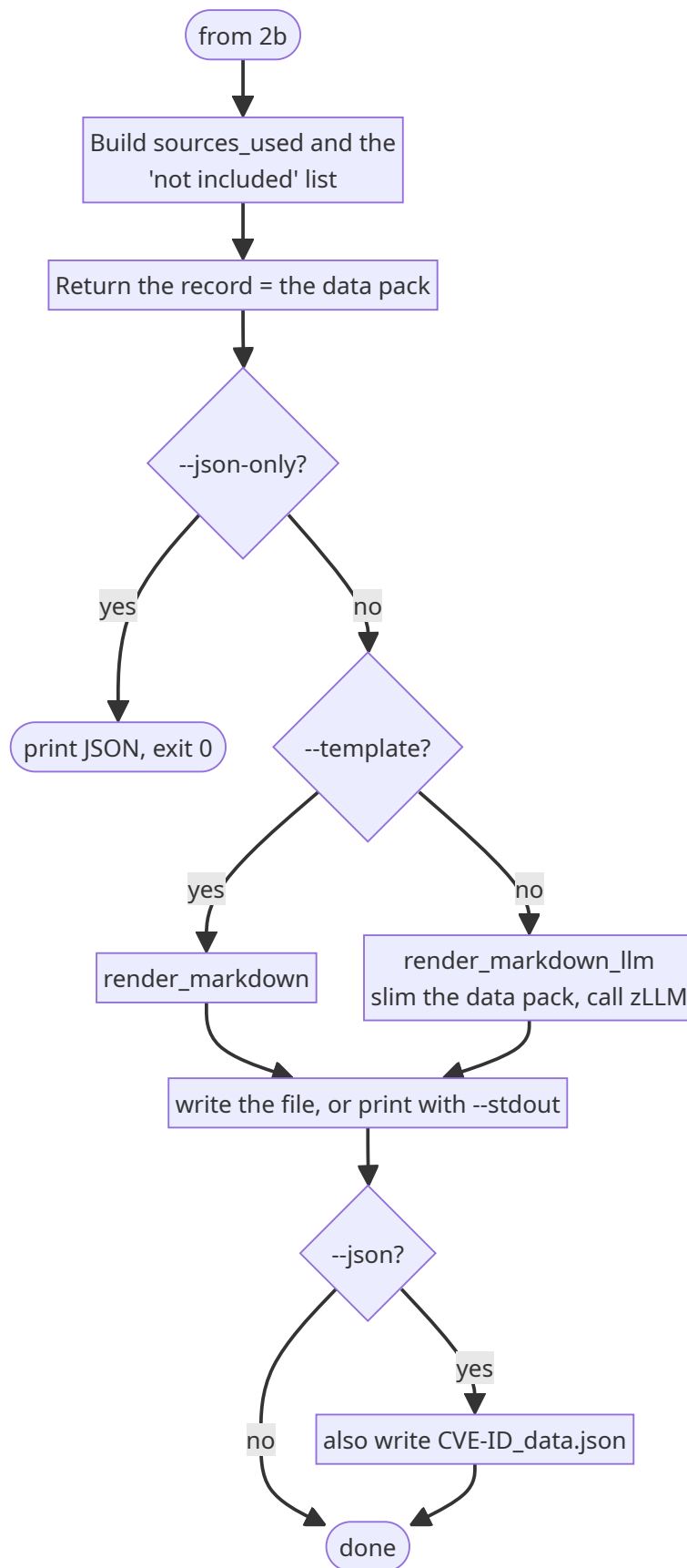
2a. Interpreting the record, patches and functions



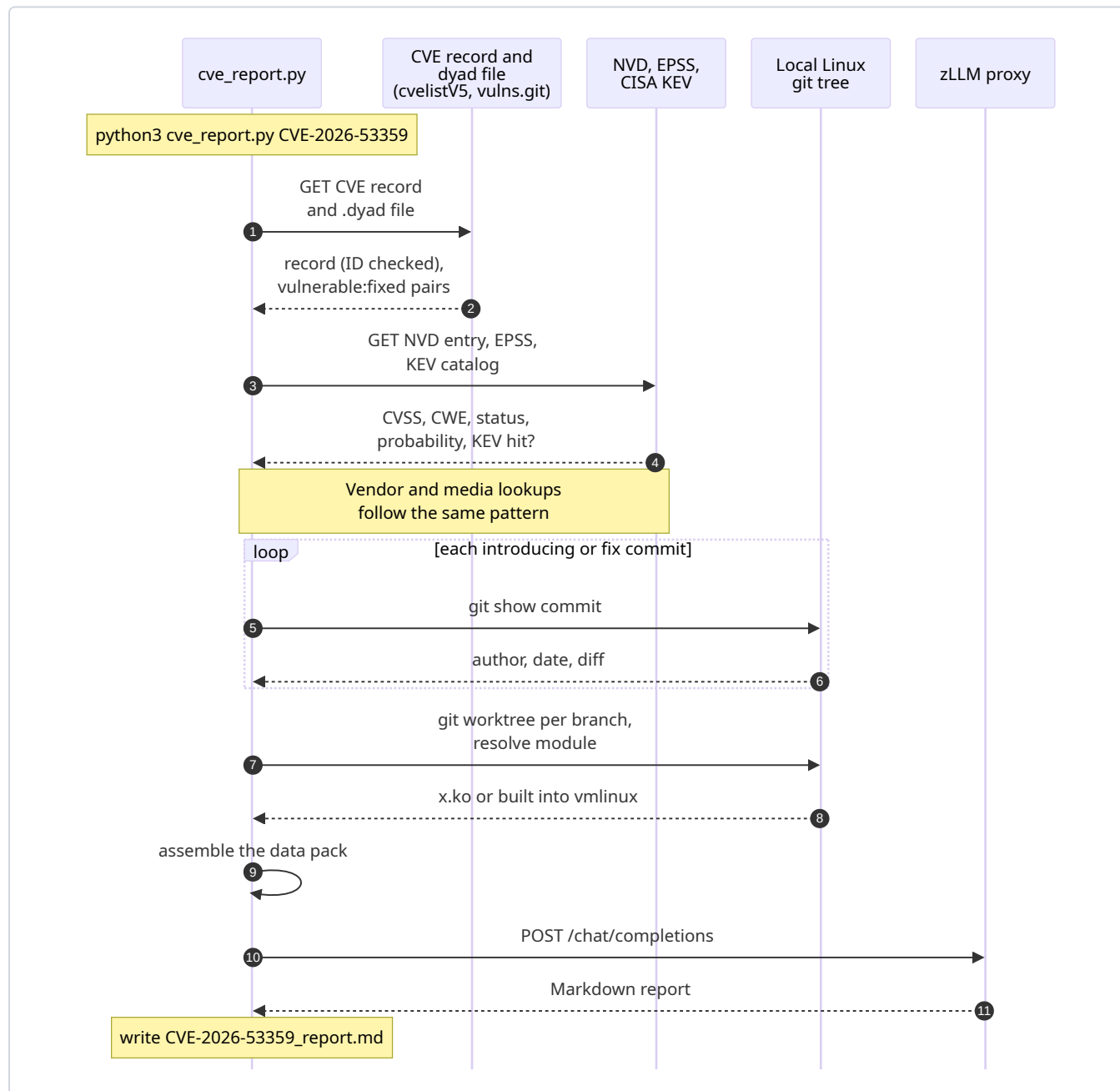
2b. Modules and fleet exposure



2c. Assembling and writing

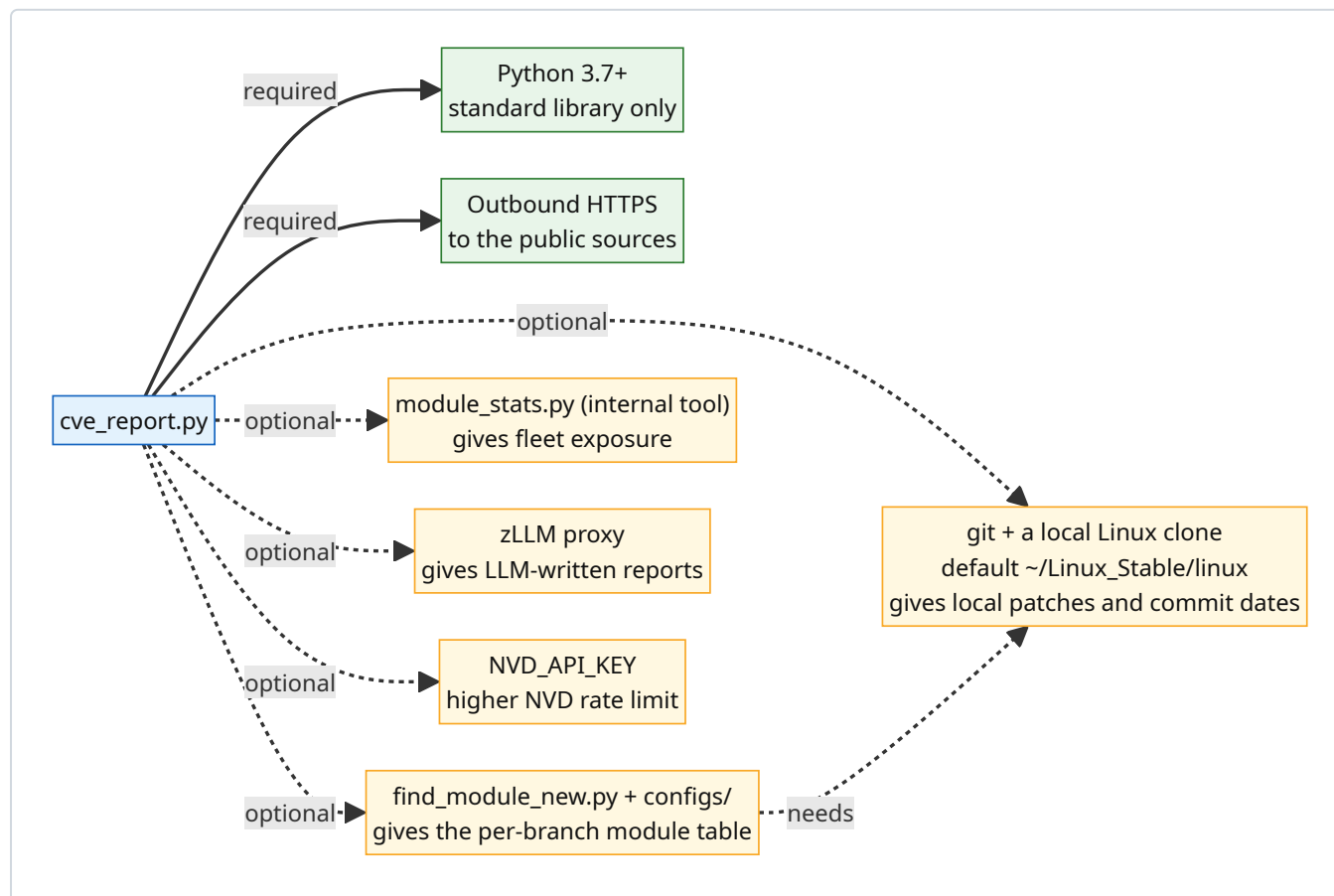


3.4 One run as a conversation



3.5 Dependencies

What is required and what is optional



Python standard-library modules

Module	Used for
<code>argparse</code>	Command-line flags
<code>urllib.request</code> , <code>urllib.error</code> , <code>urllib.parse</code>	All HTTP (GET for data, POST for the LLM), URL quoting
<code>json</code>	Parse API responses; build the data pack and the LLM request
<code>re</code>	Pattern matching: dyad lines, patch headers, Makefiles, HTML blocks, CVE-ID validation
<code>os</code> , <code>sys</code>	Paths, environment variables, stderr, exit codes
<code>subprocess</code>	Running <code>git show</code> and <code>module_stats.py</code> (always as an argument list, never through a shell)
<code>time</code>	Retry sleeps and cache age
<code>gzip</code>	Decompressing gzip responses (in practice the Debian dump, the only request that asks for gzip)
<code>datetime</code>	UTC generation timestamp, date formatting
<code>__future__.annotations</code>	Modern type-hint behaviour (this is what sets the 3.7+ floor)

`find_module_new.py` (imported lazily, only when module resolution runs) additionally uses `concurrent.futures`, `shutil`, `tempfile` and `uuid` — all standard library as well.

Non-Python things the tool talks to

Dependency	Needed for	If missing
Internet (HTTPS, outbound)	All data sources	A source that cannot be reached is skipped with a stderr note; if the CVE record cannot be fetched the run stops
<code>git</code> executable	Local patch reading; <code>find_module_new.py</code> worktrees	Patches fall back to <code>git.kernel.org</code> ; module resolution is skipped
Linux clone at <code>--linux-src</code> (default <code>~/Linux_Stable/linux</code>) with local branches <code>linux-5.10.y</code> , <code>linux-6.1.y</code> , <code>linux-6.12.y</code>	Local patches; module resolution	Patches fetched remotely; no per-branch module table
<code>configs/config-<major.minor>.<patch></code> files (e.g. <code>config-6.12.90</code>)	Choosing which <code>.config</code> to test each branch against	A branch with no matching config is silently skipped
<code>find_module_new.py</code> next to the script	The Makefile/Kconfig resolution engine	<code>ImportError</code> is caught; module resolution silently returns nothing
<code>module_stats.py</code> (<code>--module-stats-script</code>)	Fleet exposure	Skipped; reason logged
zLLM proxy at <code>ZLLM_BASE_URL</code>	LLM-written report	Run stops with an error and the hint <code>zllm start</code> — <i>by design there is no silent fallback</i> ; use <code>--template</code>

Environment variables

Variable	Default	Purpose
<code>NVD_API_KEY</code>	none	Sent as the <code>apiKey</code> header to NVD to raise its rate limit
<code>ZLLM_BASE_URL</code>	<code>http://127.0.0.1:8787/v1</code>	Where the OpenAI-compatible proxy listens
<code>ZLLM_API_KEY</code>	<code>unused</code>	Sent as <code>Authorization: Bearer ...</code> (the proxy ignores it unless configured)
<code>ZLLM_MODEL</code>	<code>gpt-5.4</code>	Model id sent to the proxy (<code>--llm-model</code> overrides)
<code>TMPDIR</code>	<code>/tmp</code>	Parent of the cache folder and of the temporary worktrees

Network hosts to allow through a firewall

`raw.githubusercontent.com`, `cveawg.mitre.org`, `git.kernel.org`, `services.nvd.nist.gov`, `api.first.org`, `www.cisa.gov`, `security-tracker.debian.org`, `www.exploit-db.com`, `tuxcare.com`, `feedly.com`, `www.thehackerwire.com`, `thehackernews.com`, `cybersecuritynews.com`, `securityonline.info` — plus the LLM proxy (local by default).

3.6 Catalogue of data sources

#	Source	Run by	What the tool takes from it	Interface	Cached
1	CVEProject/cvelistV5 (raw file on GitHub)	The CVE Program	The CVE record: title, description, files, functions, CNA/ADP CVSS, CWE, SSVC, CPE ranges, references	Raw JSON file by path	No

#	Source	Run by	What the tool takes from it	Interface	Cached
1b	cveawg.mitre.org (CVE Services API)	The CVE Program	Same record, used as a fallback	REST JSON	No
2	kernel vulns.git <code>.dyad</code>	kernel.org CNA team	Vulnerable → fixed commit/release pairs per branch; unfixed (EOL) branches	Plain text over git web	No
3	NVD 2.0 API	NIST	CVSS, CWE, <code>vulnStatus</code> , extra references	REST JSON (optional API key)	No
4	FIRST EPSS API	FIRST.org	Exploit probability and percentile	REST JSON	No
5	CISA KEV catalogue	CISA	"Known exploited?" plus due date	One JSON file, searched locally	6 h
6	git.kernel.org (stable tree)	kernel.org	Patches (fallback to local tree); directory <code>Makefile s</code>	Plain text over git web	No
7	Local Linux clone	You	Patches via <code>git show</code> , commit dates , branch Makefiles/Kconfig	<code>git</code> subprocess	—
8	<code>find_module_new.py</code> + <code>configs/</code>	You	File → module/built-in/not-compiled, per branch	Python import	—
9	<code>module_stats.py</code>	Your fleet team	Per-region counts of servers loading the module	Subprocess, text table	—
10	Debian Security Tracker	Debian project	Per-suite status and fixed package version	One big JSON dump, searched locally	24 h
11	TuxCare CVE tracker	TuxCare (commercial vendor)	Debian ELS fixes and KernelCare (live-patch) state	HTML page, regex extraction	No
12	Exploit-DB	OffSec	Public exploit entries	Undocumented search endpoint (JSON)	No
13	linuxkernelcves.com data	Community (<code>nluedtke</code>)	Colloquial name (e.g. "Dirty Pipe"), affected / last-vulnerable versions	One ~6 MB JSON file	24 h
14	Feedly <code>/cve/<id></code>	Feedly (commercial)	Trending flag, mention count, up to 5 article links	JSON embedded in HTML	No
15	The Hacker Wire	Third-party site	Per-CVE headline/description	<code>ld+json</code> block embedded in HTML	No
16	The Hacker News	Third-party news site	Up to 3 article hits	Blogger public feed	No
17	CyberSecurityNews, SecurityOnline	Third-party news sites	Up to 3 article hits each	WordPress REST search	No
—	zLLM proxy	You / your team	Writes the prose of the report	<code>POST /chat/completions</code>	—
—	Qualys blog, BleepingComputer, Ubuntu, SUSE	—	Links only (no public/no-auth per-CVE API)	—	—
—	Red Hat, Arch Linux, OSV.dev	—	Fetch/summarise code exists but is not called (see 5.8)	—	—

3.7 What happens when a source is missing?

Missing / failing	Effect on the report	Console message
CVE record cannot be fetched	Run aborts	<code>ERROR: could not fetch a record for ...</code>
LLM proxy down (default mode)	Run aborts	<code>ERROR: could not reach zLLM proxy ... Try: zllm start</code>
dyad missing (or not a kernel CVE)	Version table falls back to the CVE record's ranges; no fix/EOL table	<code>- not included: kernel vulns.git (dyad) (...)</code>
NVD / EPSS missing	Those rows are blank; CVSS still comes from the CNA/ADP	<code>- not included: NVD 2.0 API (...)</code>
Local Linux tree missing	Patches fetched from git.kernel.org; commit dates blank ; no per-branch module table	<code>- not included: Per-branch module resolution (local Linux source tree not available)</code>
<code>module_stats.py</code> missing	No fleet exposure	<code>- not included: Fleet module-usage stats (module_stats.py not found)</code>
A vendor / media site down	That row/section absent or "no coverage found"	<code>- not included: <site> (site unreachable)</code>
Cache folder not writable	Slower (re-downloads), still works	none

Part 4 — Code walkthrough (Intermediate)

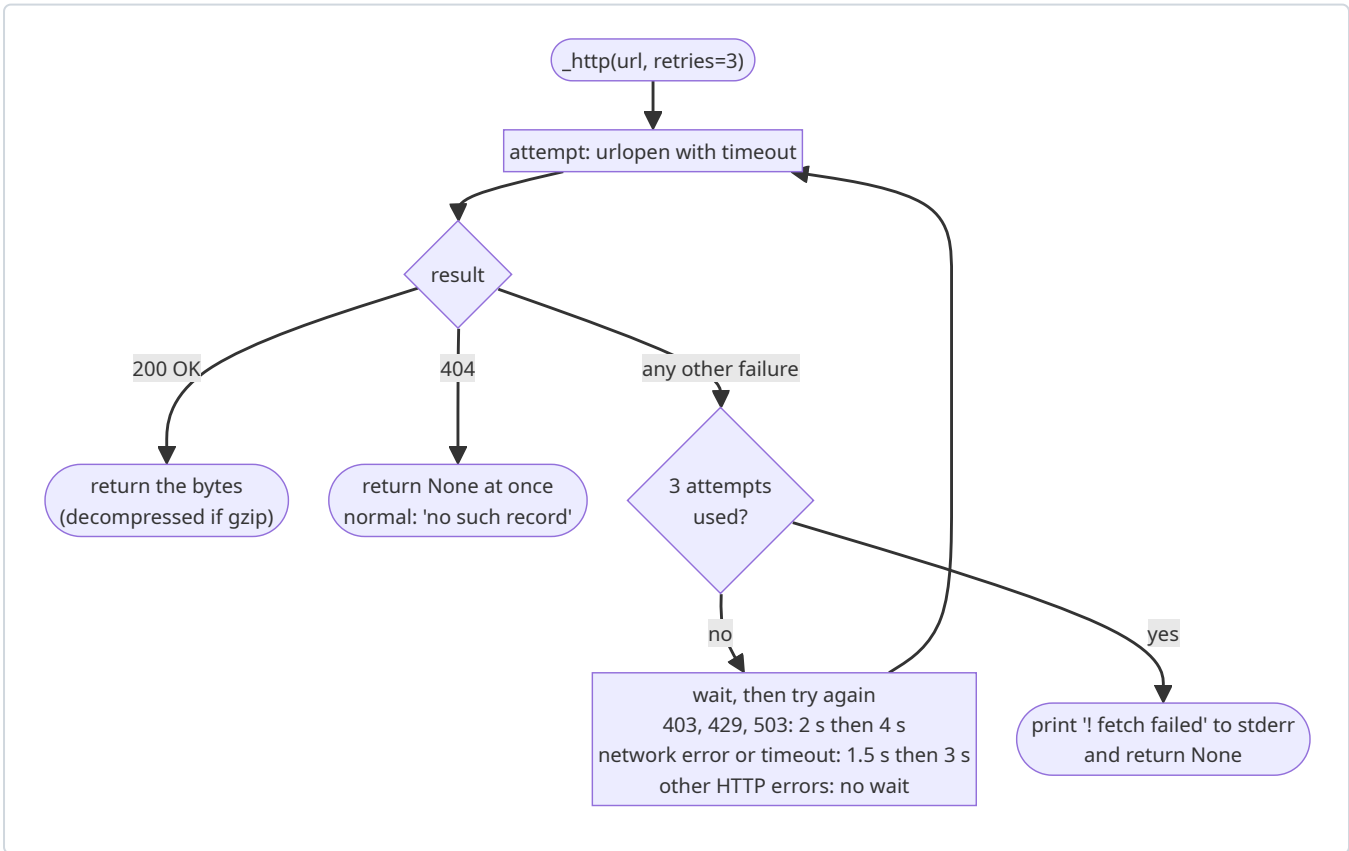
Function names are links to the code at commit `7e38fe1`. Line numbers will drift as the file changes — search by name.

4.1 Constants and configuration

Constant	Value / meaning
<code>UA</code>	<code>cve-report/1.1</code> (+ <code>https://github.com/CVEProject/cvelistV5</code>) — the User-Agent sent to every site
<code>KERNEL_GIT</code> , <code>VULNS_GIT</code>	git.kernel.org URLs for the stable tree and the <code>vulns.git</code> repo
<code>CACHE_DIR</code>	<code>\$TMPDIR/cve_report_cache</code> (default <code>/tmp/cve_report_cache</code>)
<code>CVE_RE</code>	<code>^CVE-\d{4}-\d{4,}\$</code> (case-insensitive) — the input validator
<code>LINUX_SRC_DEFAULT</code>	<code>~/Linux_Stable/linux</code>
<code>CONFIGS_DIR_DEFAULT</code>	<code>configs/</code> next to the script
<code>MODULE_RESOLVE_BRANCHES</code>	<code>["linux-5.10.y", "linux-6.1.y", "linux-6.12.y"]</code> — the branches your fleet runs
<code>MODULE_STATS_SCRIPT_DEFAULT</code>	<code>~/Server-Modules/Scripts/module_stats.py</code>
<code>ZLLM_*</code>	Proxy URL, key, model, system prompt, user-prompt template
<code>ZLLM_MAX_DATA_PACK_CHARS</code>	<code>350000</code> — the size budget for what is sent to the LLM
<code>*_PAGE</code> , <code>*_API</code> , ...	URL patterns with a <code>%s</code> for the CVE ID
<code>NOT_A_FUNCTION</code>	Regex of macro-like names to ignore when guessing changed functions (<code>EXPORT_SYMBOL...</code> , <code>MODULE_...</code> , <code>DEFINE_...</code>)
<code>FEEDLY_NOISY_CHATTER_HOSTS</code>	Hosts whose links are dropped from Feedly results (Google News redirects, X/Twitter mirrors)
<code>CWE_KEYWORDS</code>	Keyword → CWE table. Currently unused (see 5.8)

4.2 The HTTP layer

Every network read goes through `_http`^{L234}. The two wrappers `_http_json`^{L264} and `_http_text`^{L274} decode the bytes.



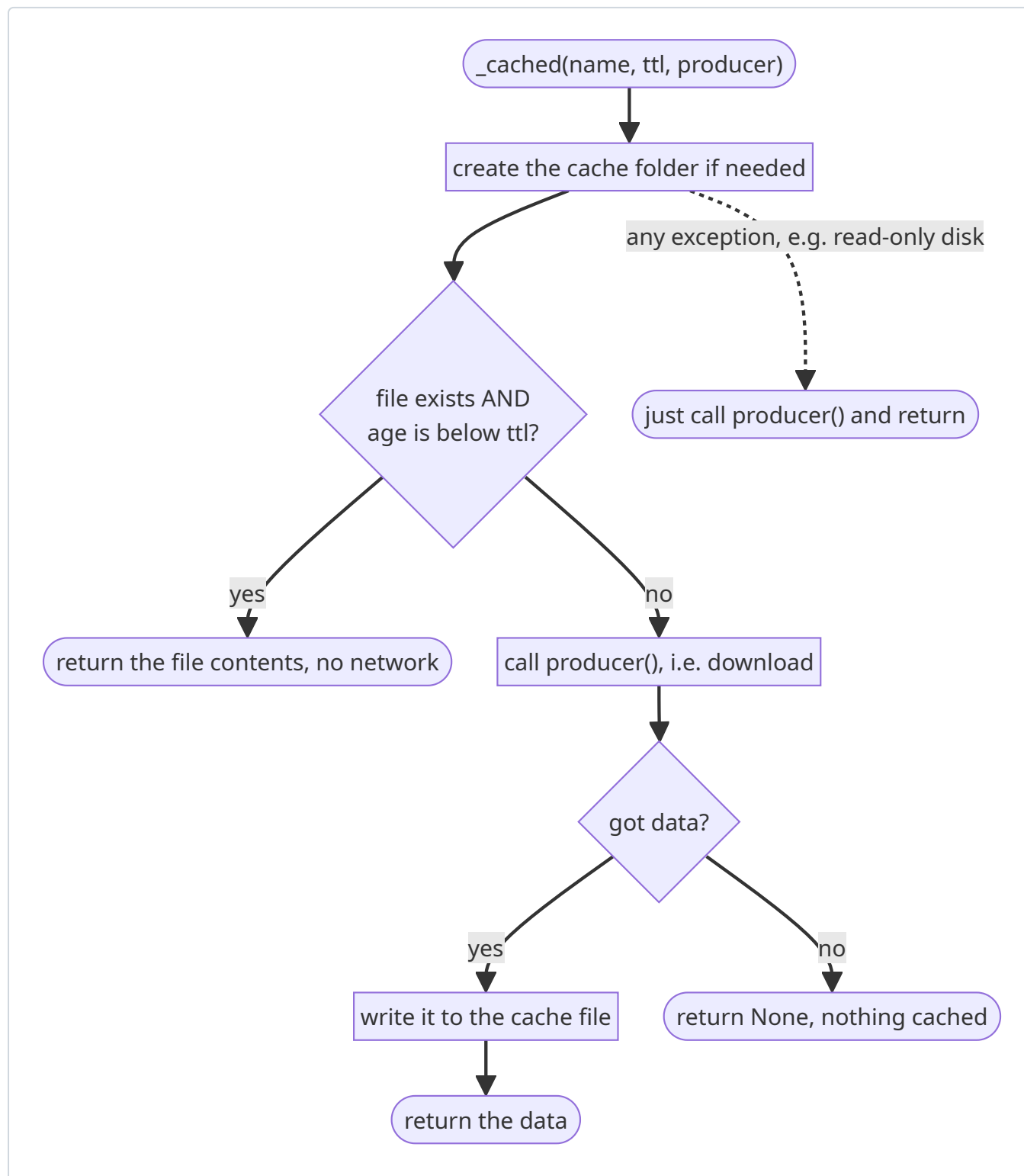
Key behaviours:

- **404 is silent.** Most sources legitimately have no entry for most CVEs; that is "no data", not "error".
- **Failures never raise.** `_http` returns `None`, so one dead website cannot crash the run.
- Worst case for one URL is roughly `3 × timeout + sleeps` (e.g. about 95 s with a 30 s timeout).
- The tool **does not** send `Accept-Encoding: gzip` by default; only the Debian dump request asks for it (12 MB compressed vs. roughly 80 MB raw, per the code comment).

`llm_chat`^{L279} is separate on purpose: it **does** raise (`SystemExit` with an actionable message), because without the LLM there is no report to write and silently falling back would hide a configuration problem.

4.3 The cache

`_cached(name, ttl, producer)`^{L320} wraps the three big downloads.



Cache file	TTL	Why cached
<code>kev.json</code>	6 h	The KEV catalogue is one file for all CVEs
<code>debian_tracker.json</code>	24 h	No per-CVE endpoint exists; the dump is large
<code>linux_kernel_cves.json</code>	24 h	One ~6 MB file for all CVEs

To force fresh data: `rm -rf "${TMPDIR:-/tmp}/cve_report_cache"`.

4.4 Building the CVE-list URL

CVE records live in folders grouped by year and by "thousands bucket". `_cve_parts`^{L340} computes the bucket:

```
CVE-2026-53359    year = 2026    number = 53359    bucket = number without last 3 digits + "xxx" = 53xxx
CVE-2026-100234   year = 2026    number = 100234   bucket = 100xxx
CVE-2026-512      year = 2026    number = 512      bucket = 0xxx      (3 digits or fewer)
```

```
https://raw.githubusercontent.com/CVEProject/cvelistV5/main/cves/2026/53xxx/CVE-2026-53359.json
```

`cvelist_url`^{L349} gives the raw-file URL; `cvelist_blob_url`^{L357} gives the human-readable GitHub page that is added to the References section.

4.5 The fetchers, one by one

Return-value convention (important when modifying code)

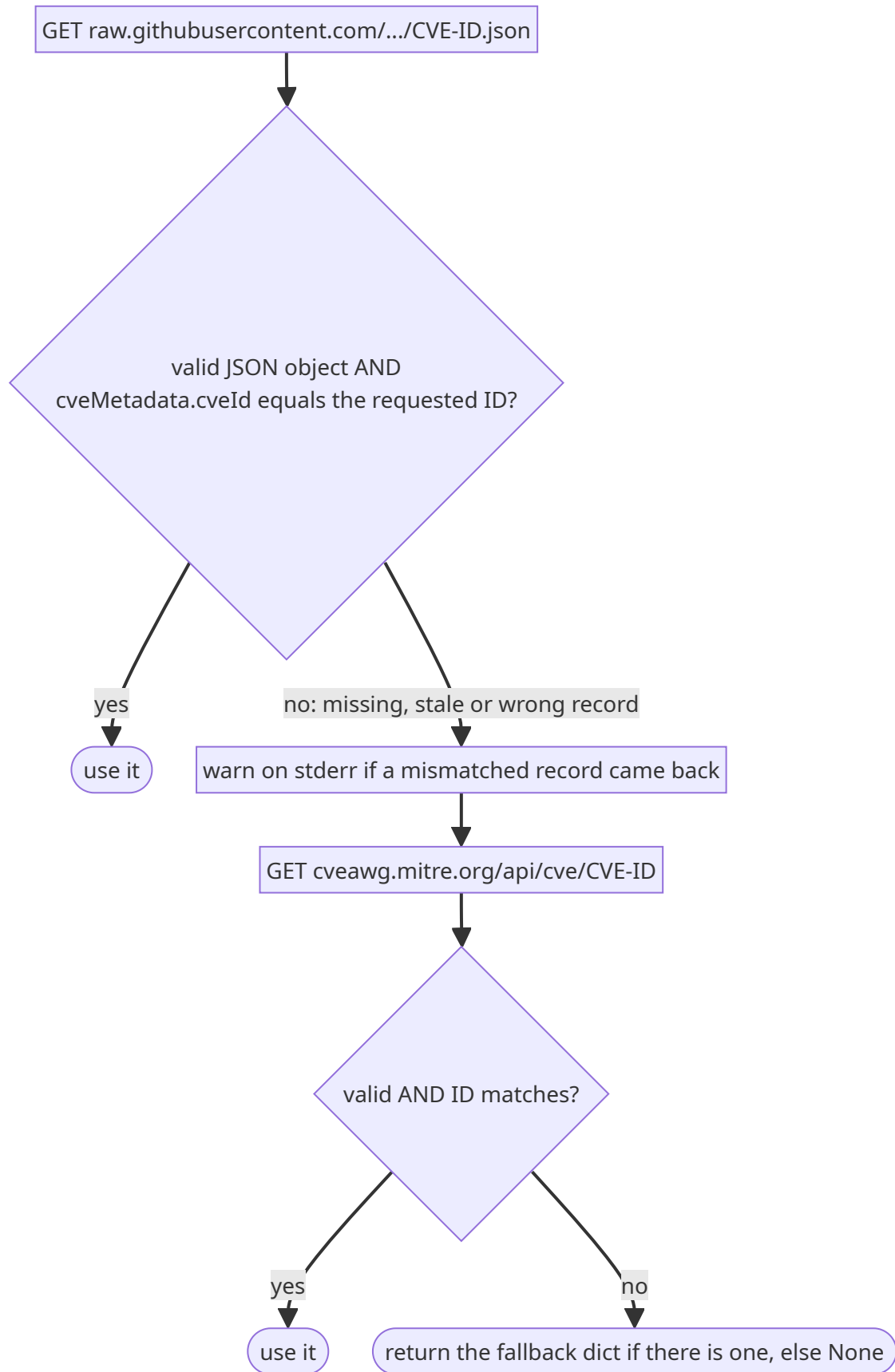
Most fetchers return `None` for *both* "nothing found" and "could not reach". Two families keep the two apart:

Function	Success	Reached, but nothing found	Could not reach
<code>fetch_exploitdb</code> ^{L501}	list of rows	<code>[]</code>	<code>None</code>
<code>fetch_wp_coverage</code> ^{L513} , <code>fetch_hackernews_coverage</code> ^{L536}	list of hits	<code>[]</code>	<code>None</code>
everything else	dict / text	<code>None</code>	<code>None</code>

`build_record` uses the difference to write accurate "not included" reasons and to print "no coverage found" vs. nothing.

The core fetchers

Function	Request	Notes
<code>fetch_cvelist</code> ^{L369}	GitHub raw file, then <code>cveawg.mitre.org/api/cve/<id></code>	Validates that <code>cveMetadata.cveId</code> equals the requested ID because the raw CDN "can serve stale blobs" (see diagram below)
<code>fetch_dyad</code> ^{L391}	<code>.../vulns.git/plain/cve/published/<year>/<ID>.dyad</code>	Only called when <code>is_kernel</code>
<code>fetch_nvd</code> ^{L400}	<code>services.nvd.nist.gov/rest/json/cves/2.0?cveId=<ID></code>	Adds <code>apiKey</code> header if <code>NVD_API_KEY</code> is set; returns <code>vulnerabilities[0].cve</code>
<code>fetch_epss</code> ^{L410}	<code>api.first.org/data/v1/epss?cve=<ID></code>	Returns <code>{"epss": "0.0017...", "percentile": "0.062...", "date": ...}</code> — strings, converted with <code>float()</code> when rendered
<code>fetch_kev</code> ^{L418}	CISA JSON feed (cached 6 h)	Downloads the whole catalogue, scans <code>vulnerabilities[]</code> for <code>cveID</code>



Vendor and community fetchers

Function	What it does	Gotchas
<code>fetch_debian_entry</code> ^{L480}	Downloads (cached) the whole Debian tracker JSON, looks under package <code>linux</code> first, then every package	The only way — Debian has no per-CVE endpoint
<code>fetch_linuxkernelcves</code> ^{L464}	Downloads (cached) the community JSON, returns <code>data[cve_id]</code>	Docstring: "community-maintained, no accuracy guarantee"
<code>fetch_exploitdb</code> ^{L501}	<code>exploit-db.com/search?cve=<ID></code> with header <code>X-Requested-With: XMLHttpRequest</code>	Endpoint is undocumented; the code relies on it returning JSON
<code>fetch_tuxcare</code> ^{L559}	Regex 1: embedded <code>const allData = [...]</code> → Debian <i>ELS</i> fixes with status <code>released</code> . Regex 2: the "KernelCare state" HTML table → KCARE fixes	The most fragile fetcher — it reads HTML structure
Red Hat / Arch / OSV	<code>fetch_redhat</code> ^{L434} , <code>fetch_archlinux</code> ^{L441} , <code>fetch_osv</code> ^{L457}	Written and documented but never called

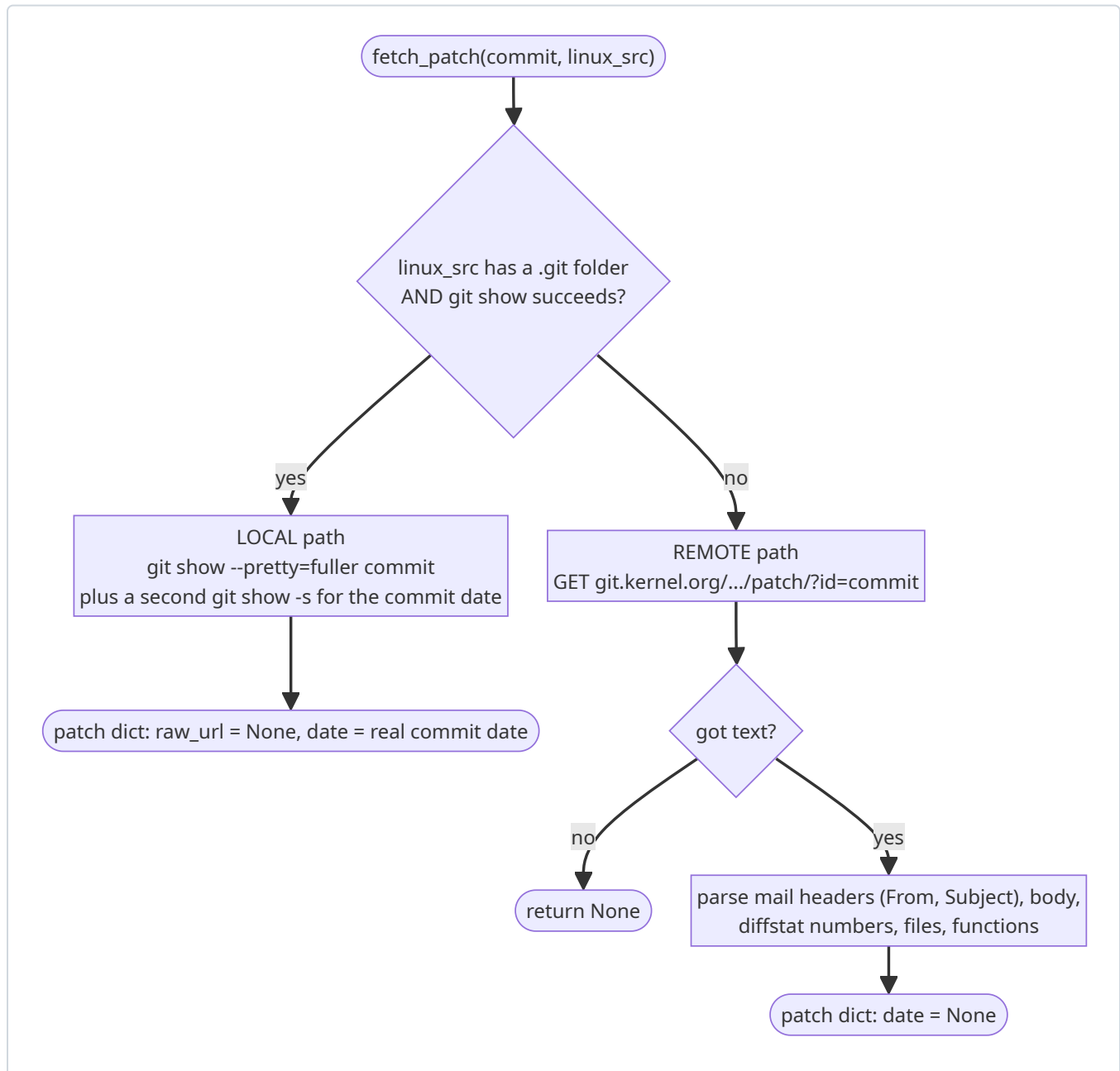
Media fetchers

Function	Technique	Returns
<code>fetch_feedly</code> ^{L621}	Extract the <code><script id="__NEXT_DATA__"></code> JSON from the page; keep CVSS estimate, <code>trending</code> , <code>patched</code> , total mentions and ≤ 5 article links (noisy hosts removed)	dict or <code>None</code>
<code>fetch_hackerwire</code> ^{L594}	Extract the <code>application/ld+json</code> block and read the <code>TechArticle</code> node. The site returns HTTP 200 even for unknown CVEs, so <i>no ld+json block</i> means "not tracked"	dict or <code>None</code>
<code>fetch_hackernews_coverage</code> ^{L536}	Blogger feed search <code>.../feeds/posts/default?q=<term>&alt=json</code> ; tries the CVE ID, then the colloquial name	list / <code>[]</code> / <code>None</code>
<code>fetch_wp_coverage</code> ^{L513}	WordPress REST <code>.../wp-json/wp/v2/posts?search=<term></code> ; same two-term logic	list / <code>[]</code> / <code>None</code>

Important about media results. They are **keyword searches**, so a hit means "an article mentions that text", not "an article about this vulnerability". In the real sample run, Feedly's list included unrelated vendor bulletins and a SecurityOnline hit titled for a different CVE. Treat section 9 as leads, never as evidence.

4.6 Reading a commit: local first, network second

The tool needs, for each relevant commit: author, **commit date**, subject, body, changed files, +/– line counts, and the names of changed functions. `fetch_patch`^{L758} tries two paths.



Why is the remote `date` left empty? The patch e-mail's `Date:` header is the *author* date (when the change was first written). The tool's rule (stated in the README) is to report the **commit date** only, and that is available only from a git tree. So if a commit is not in your local clone, its date is blank rather than wrong.

How "changed functions" are guessed. Git prints, after each hunk's `@@ ... @@`, the nearest preceding "function context" line. The tool reads those for `.c` / `.S` files only:

```
@@ -1234,7 +1234,9 @@ static struct kvm_mmu_page *kvm_mmu_get_child_sp(struct kvm_vcpu *vcpu,    ←
illustrative
└─ line ranges ─┘ ─────────── text after the second @@: git's guess of the function ───────────
```

`_func_from_context`^{L822} takes the first identifier followed by `(` that is not an ALL-CAPS macro and not matched by `NOT_A_FUNCTION`. The result is a *hint*: the report marks it "derived from diff — approximate" whenever the CNA did not list the functions itself.

4.7 Turning raw data into facts

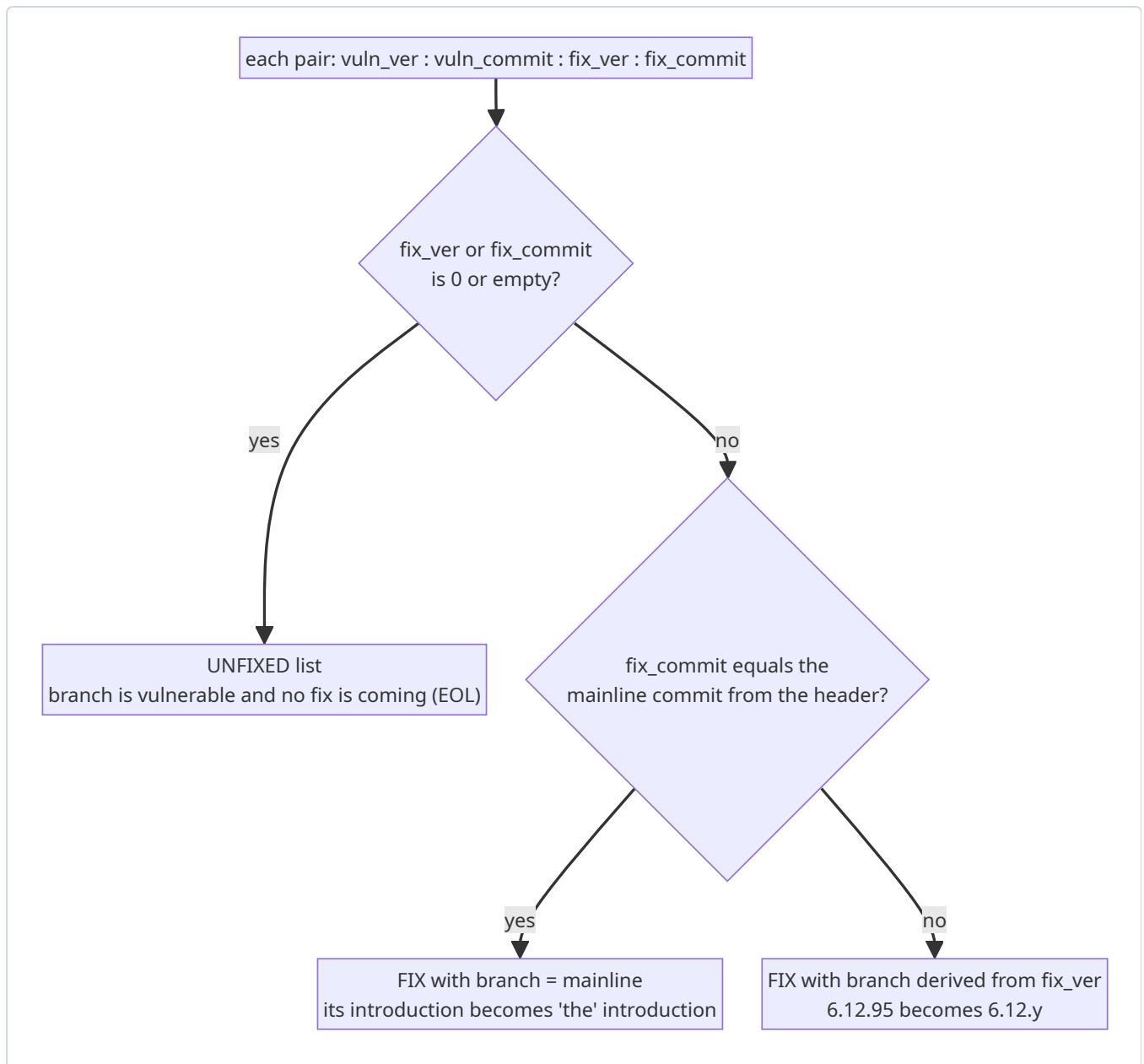
The dyad file → per-branch fix table

A real `.dyad` file (CVE-2026-53359):

```
# dyad version: 1.2.0
#      getting vulnerable:fixed pairs for git id 81ccda30b4e83d8f5cc4fd50503c44e3a33abfeb
2.6.36:2032a93d66fa...:5.15.222:0b5e8ead71e6...
2.6.36:2032a93d66fa...:6.1.177:b1337aae5e19...
2.6.36:2032a93d66fa...:6.6.144:9291654d69e0...
2.6.36:2032a93d66fa...:6.12.95:2ad3afa40ac6...
2.6.36:2032a93d66fa...:6.18.38:5e470998a23e...
2.6.36:2032a93d66fa...:7.1.3:1ae7d5a6db6c...
2.6.36:2032a93d66fa...:7.2:81ccda30b4e8...
```

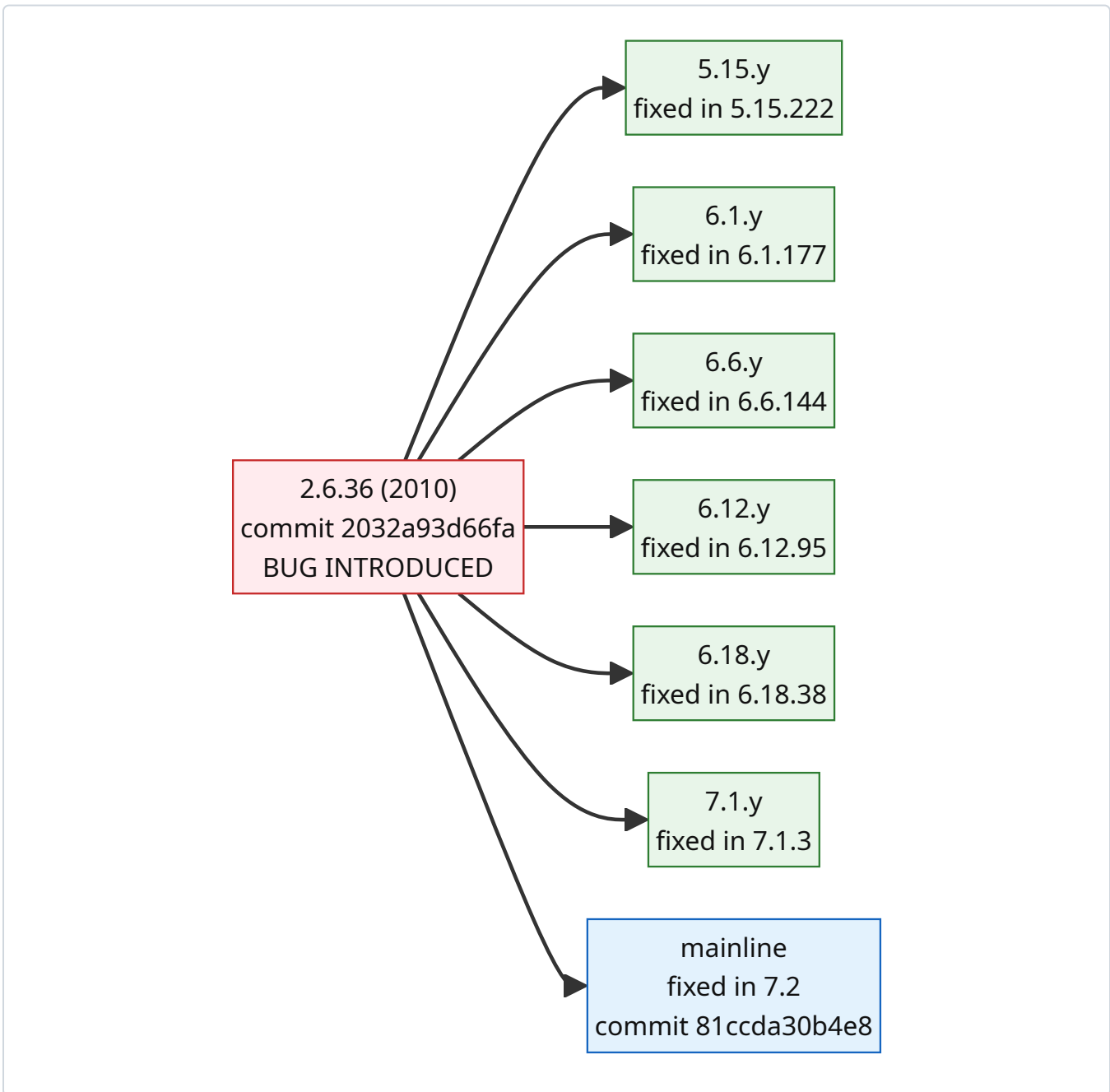
Each line is `vulnerable version : vulnerable commit : fixed version : fixed commit` (hashes shortened here with `...`). The last line's fixed commit equals the commit named in the header, so that pair is the **mainline** fix.

- `parse_dyad`^{L1064} reads the **mainline commit** from the header (`pairs for git id <hash>`), then every non-comment line with exactly four `:`-separated fields.
- `versions_from_dyad`^{L1091} classifies each pair:



- Fixes are sorted **mainline first, then stable branches from newest to oldest**.
- `intro_backported` collects any *other* introduction versions (cases where the bug was copied into older stable branches by a back-port).

Visually, for the sample CVE:



Why not just read the CVE record? The record stores versions and commits as *parallel lists*, and the code comments (and README) explain that zipping them by position breaks when a CVE has EOL branches, backports of the bug, or an unequal number of commits vs. version ranges. The dyad file states each pairing explicitly, so it is used whenever it exists. `versions_from_record`^{L1135} is only a fallback and deliberately does **not** attempt per-branch pairing.

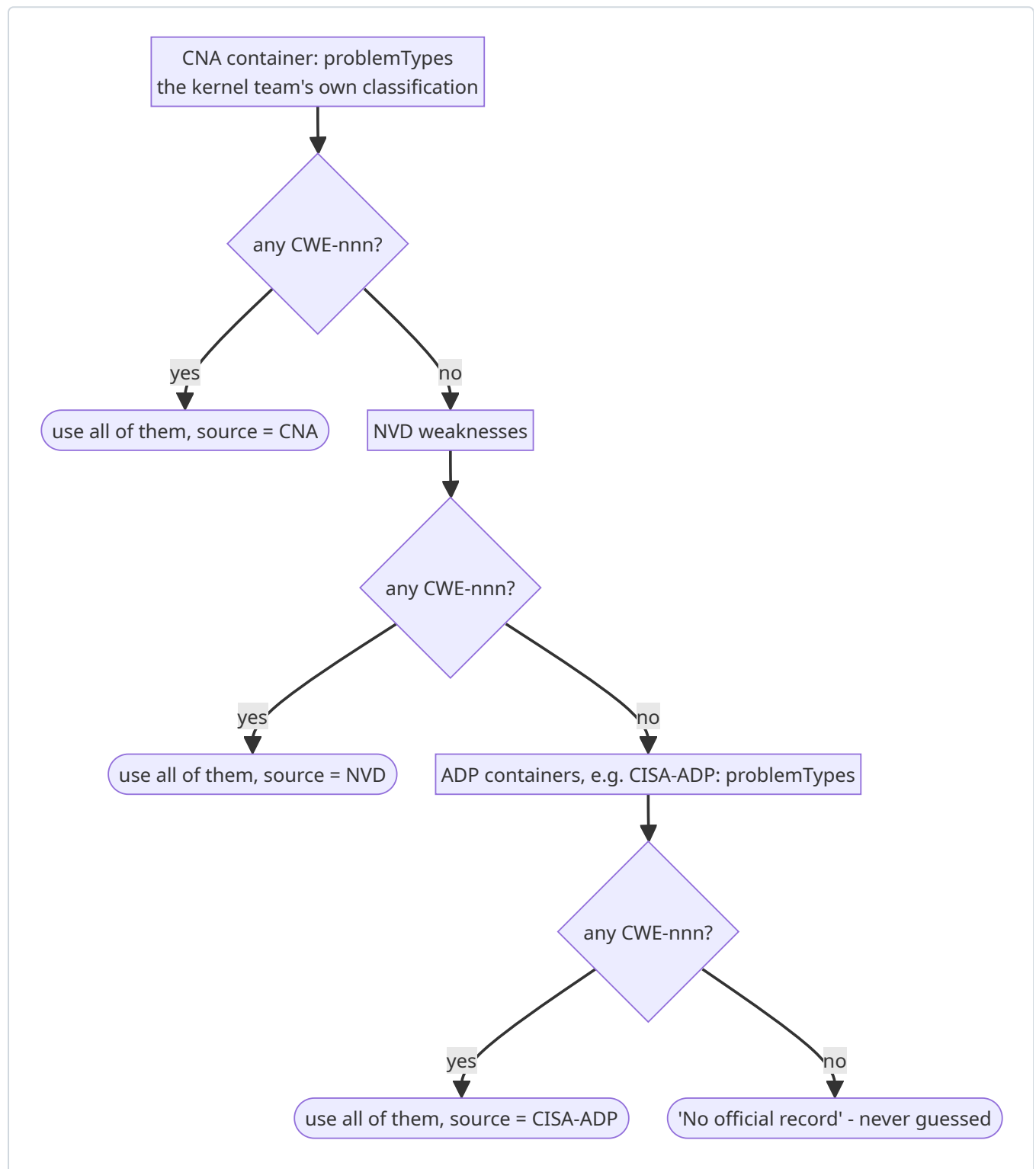
CVSS

`parse_cvss`^{L1158} collects scores from three places and **keeps them separate, labelled by source**:

1. the CNA container — `CNA (Linux)`,
2. each ADP container — `ADP (CISA-ADP)`,
3. NVD — `NVD (<source>)` (metric keys `cvssMetricV40/V31/V30/V2`).

Within each metric it looks for `cvssV4_0`, `cvssV3_1`, `cvssV3_0`, `cvssV2_0`. Duplicates (same version **and** vector) are removed, first occurrence wins. The template renderer shows the first entry as *the* CVSS and lists differing others as "CVSS (alt)".

CWE: official data only, in a fixed order



The first source with *any* CWE wins, and **all** of that source's CWEs are listed (e.g. `CWE-362`, `CWE-416`) together with the source name. There is a keyword-guessing helper (`classify_cwe`) in the file, but `parse_cwe` intentionally no longer calls it.

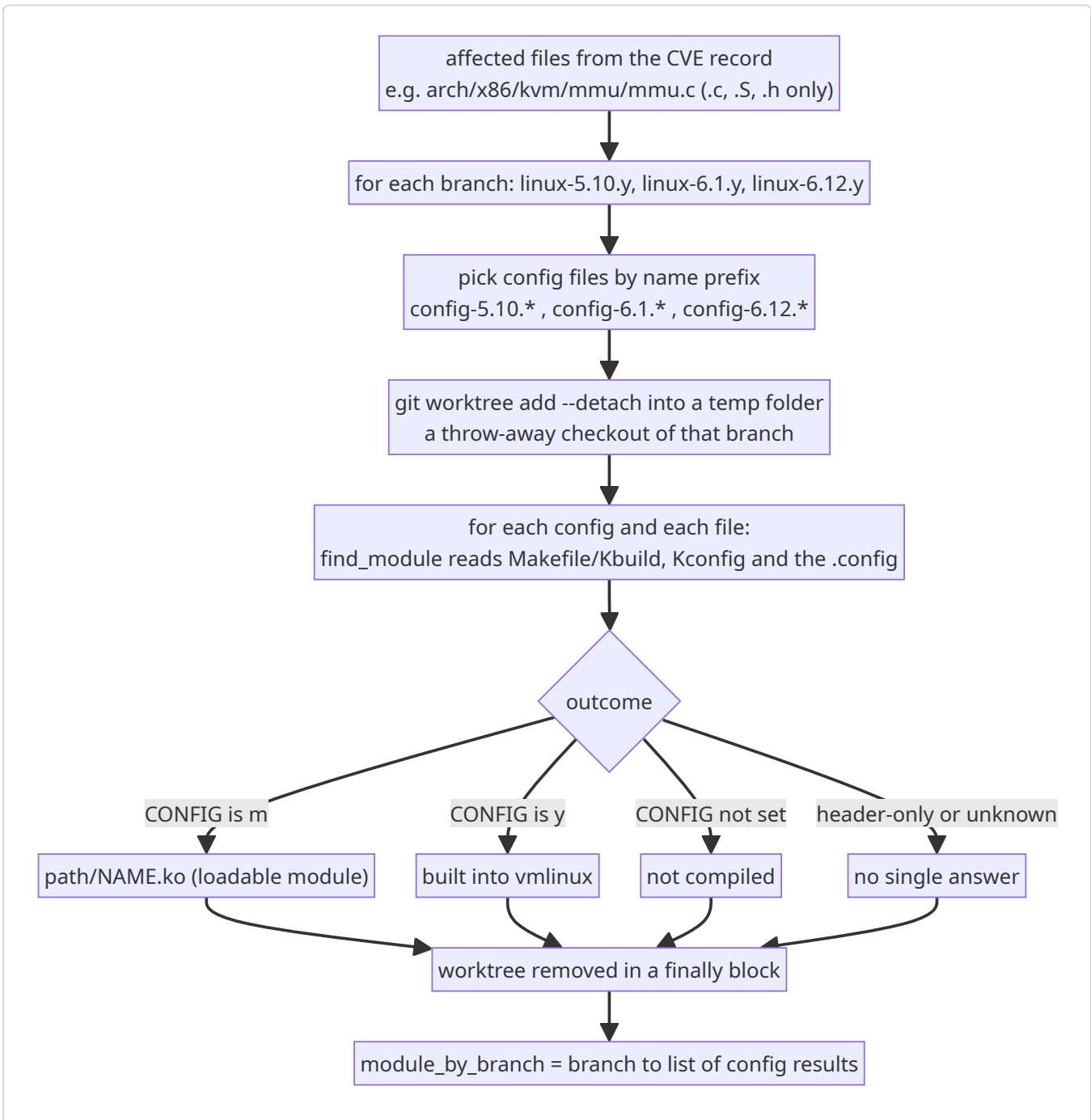
SSVC, CPE, references, titles

Function	Behaviour
<code>parse_ssvc</code> ^{L1249}	Finds an ADP metric whose <code>other.type == "ssvc"</code> ; returns <code>exploitation</code> , <code>automatable</code> , <code>technical_impact</code> , <code>role</code> and the publisher
<code>parse_cpe_ranges</code> ^{L1341}	Walks <code>cpeApplicability</code> → <code>nodes</code> → <code>cpeMatch</code> in CNA and ADP containers; keeps <code>vulnerable: true</code> ranges with inclusive/exclusive bounds; de-duplicates
<code>categorize_refs</code> ^{L1368}	Sorts every reference URL into <code>commits</code> / <code>cve_records</code> / <code>discussion</code> / <code>advisories</code> / <code>other</code> by hostname or path keyword (e.g. <code>git.kernel.org</code> → <code>commits</code> , <code>openwall.com</code> → <code>discussion</code>)
<code>derive_subsystem</code> ^{L1393}	Takes the leading <code>Name:</code> parts of the title (each ≤ 25 chars): <code>KVM: x86: Fix ...</code> → <code>KVM: x86</code> ; falls back to the first file's directory
<code>_title_from_desc</code> ^{L1409}	Kernel descriptions begin <code>...has been resolved:</code> + blank line + commit subject; the subject becomes the title
<code>summarize_debian</code> ^{L1287} , <code>summarize_exploitdb</code> ^{L1324}	Reduce big source objects to the few fields the report needs

4.8 "Which module is this?" — two mechanisms

	<code>fetch_makefile_config</code>	<code>resolve_modules_by_branch</code>
Where it reads	git.kernel.org (Makefile of the file's directory, then its parent)	Your local clone , per branch, in a temporary worktree
Branch awareness	None — reads the default branch (current development tree)	Yes — <code>linux-5.10.y</code> , <code>linux-6.1.y</code> , <code>linux-6.12.y</code>
Uses your <code>.config</code> ?	No	Yes — decides built-in vs. module vs. not compiled
Method	Regex heuristics over Makefile lines (<code>foo-y</code> , <code>foo-objs</code> , <code>obj-\$(CONFIG_X) += foo.o</code>)	<code>find_module_new.py</code> — a dedicated Makefile + Kbuild + Kconfig resolver
Output	<code>modules</code> , <code>config</code> , <code>makefiles</code>	<code>module_by_branch</code>
Reliability	Best effort ("heuristic")	The rigorous one for your fleet

`resolve_modules_by_branch` ^{L944} in detail:

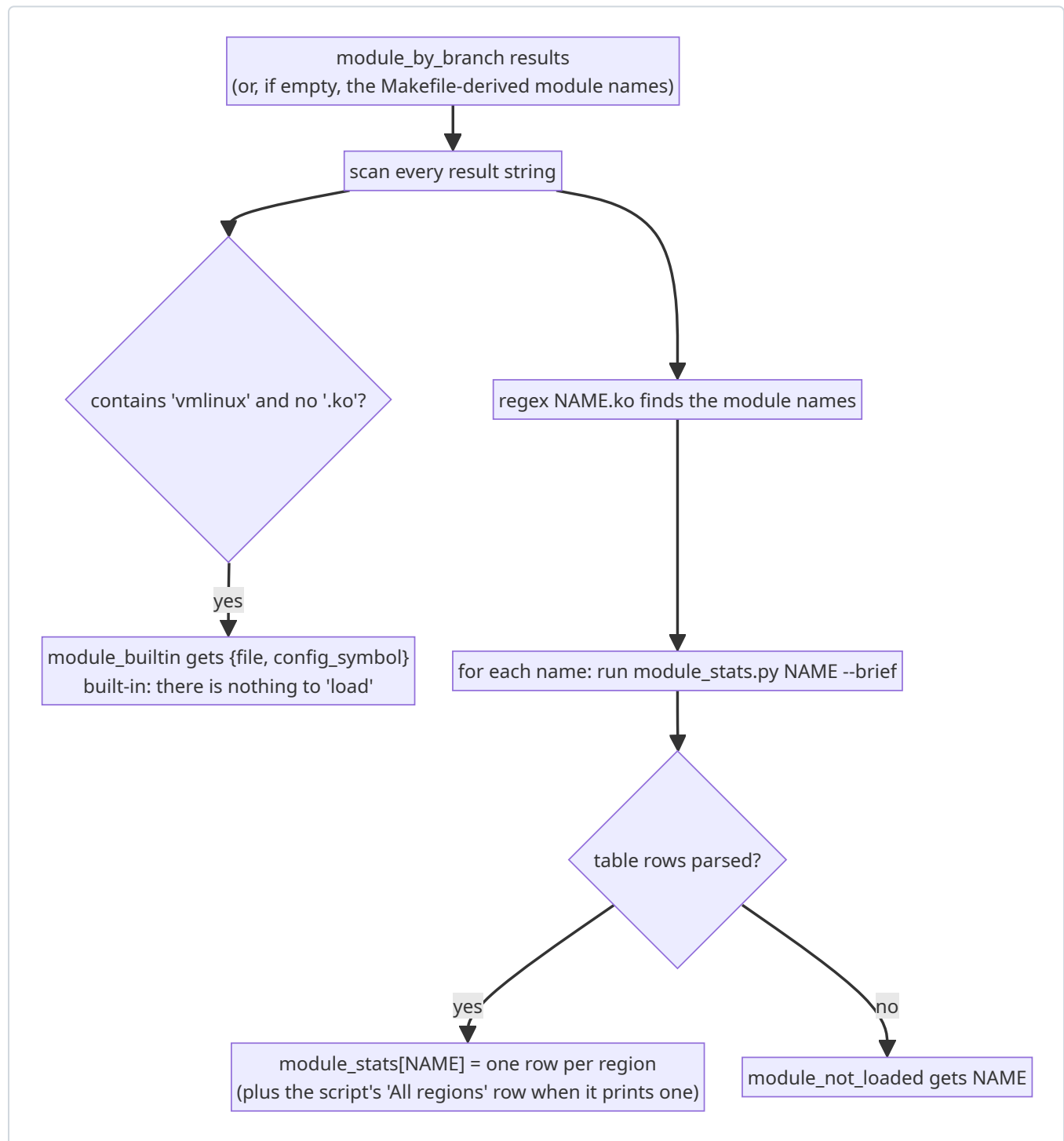


Result strings come from `format_outcome`, e.g. (real data for `net/ipv6/ah6.c`, `CONFIG_INET6_AH`):

Branch	Config used	File	CONFIG_ symbol	Result
linux-5.10.y	config-5.10.106 / .191 / .226	net/ipv6/ah6.c	CONFIG_INET6_AH	net/ipv6/ah6.ko
linux-6.1.y	config-6.1.112 / .123	same	same	net/ipv6/ah6.ko
linux-6.12.y	config-6.12.74 / .90	same	same	net/ipv6/ah6.ko

Cost. `cve_report.py` asks for a *full* checkout of each branch (it passes no sparse-checkout patterns), once per branch, one after another. This is expected to be the slowest optional step, which is why `--no-module-resolve` exists. If a run is killed hard (e.g. `kill -9`), a leftover worktree can remain: clean up with `git -C ~/Linux_Stable/linux worktree prune`.

4.9 Fleet exposure



`fetch_module_stats`^{L910} parses the script's text table with a regex into rows with: region, total IPs, loaded, ZServices, VMs, physical servers, KVM hosts, container hosts, containers.

The three outcomes are rendered differently:

Outcome	Meaning for the reader
<code>module_builtin</code>	Compiled into <code>vmlinux</code> . Every kernel built with that <code>CONFIG_</code> symbol contains the code. Fleet module-load statistics do not apply
<code>module_stats</code>	Loadable module; here is where it is loaded
<code>module_not_loaded</code>	Not loaded in any region — so the fleet is not exposed <i>via that module</i> (but see the caveat in 5.8, item 5)

4.10 Assembling the record and the "not included" list

At the end of `build_record`^{L1420}:

1. `sources_used` is built from what actually produced data (e.g. `CISA KEV` vs. `CISA KEV (not listed)`; `local Linux source (...)` vs. `git.kernel.org (commits)`).
2. A `skipped` list pairs each unused source with a precise reason (`disabled (--no-vendor)`, `not a Linux kernel CVE`, `site unreachable`, ...). It is printed to stderr — it is **not** in the data pack.
3. The big dict is returned.

4.11 Rendering

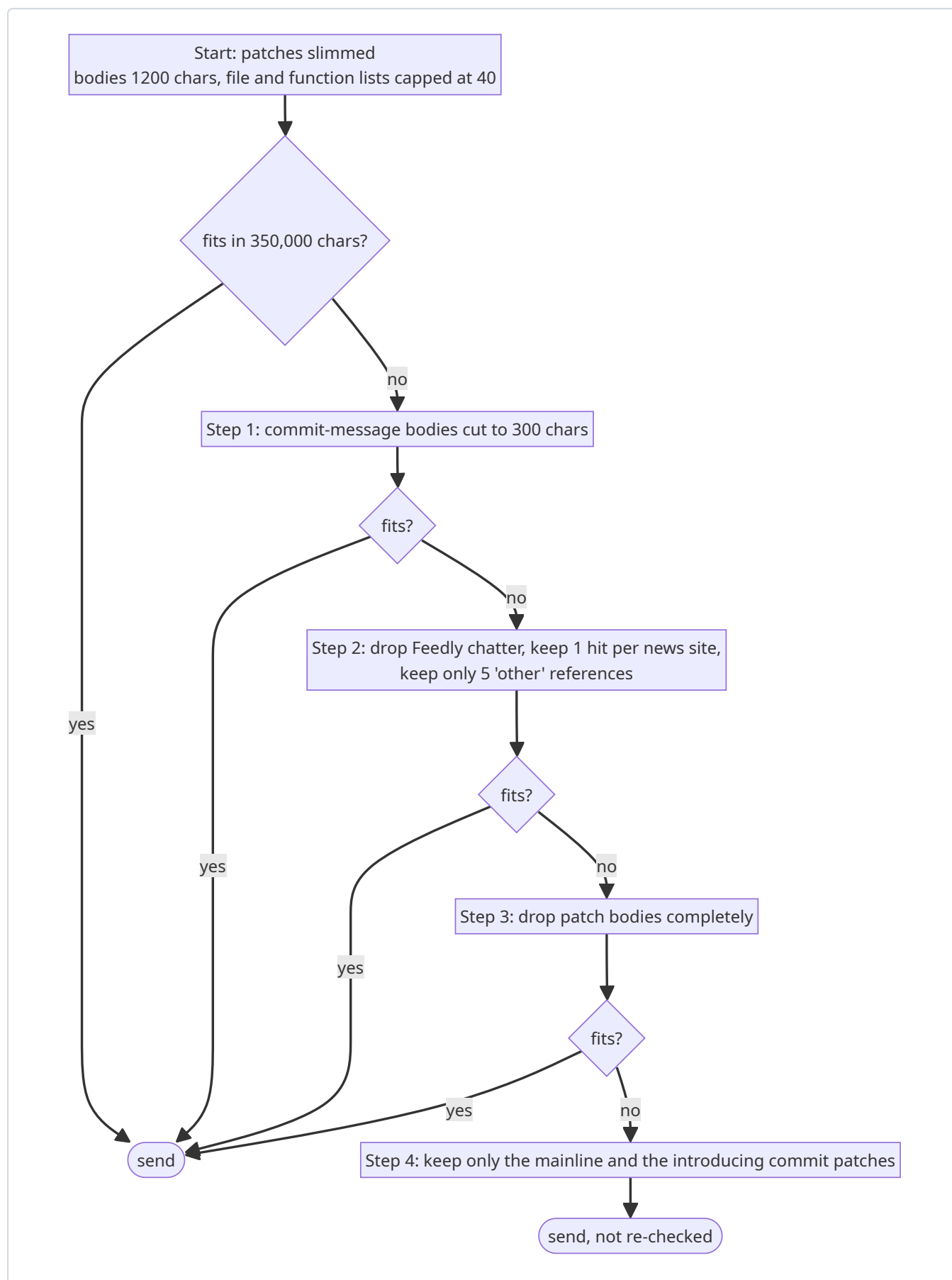
Path A — the LLM (default): `render_markdown_llm`^{L1838}

1. `_slim_data_pack`^{L1790} shrinks the record to fit a budget (below).
2. The user message is the prompt template (`ZLLM_REPORT_USER_TEMPLATE`) with the CVE ID and the pretty-printed JSON inserted.
3. `llm_chat`^{L279} POSTs `{"model", "temperature": 0.2, "messages": [system, user]}` to `<ZLLM_BASE_URL>/chat/completions` (180 s timeout).
4. The reply is checked: a `# Title` is added if missing, and a one-line `> Generated ... · sources: ...` banner is inserted under the title.

What the prompts tell the model:

Rule	Purpose
"Use ONLY the facts present in that JSON — never invent CVE numbers, commits, versions, scores, or links"	Grounding / anti-hallucination
"If a field is missing or null, say so plainly" → write <code>Not available</code> / <code>Not yet assessed</code>	No guessing
Never mention "the JSON", "data pack", or field names	The output must read as an official advisory
Fixed section order 1–10 (with 3b, TuxCare subsection, per-branch module table, fleet table, backports table with a date column)	Consistent structure
Do not include Red Hat, Ubuntu, SUSE, Arch or OSV sections	Matches the product decision
List all CVE IDs and state their source	Transparency

Keeping the data pack within the proxy's content limit — `_slim_data_pack`^{L1790} works down a ladder and stops as soon as the JSON fits in **350 000 characters**:



Core fields (CVSS, versions, description, module tables) are never trimmed.

Path B — the template: `render_markdown` ^{L1858}

A plain Python function that appends lines to a list: a fixed layout with the same ten sections, no model, fully predictable. Use it when the proxy is down, when you want byte-for-byte reproducibility, or as a **reference to compare an LLM report against**.

4.12 `main()`

`main` ^{L2266} parses flags, validates the CVE ID, calls `build_record`, and then:

Flag combination	Result
<code>--json-only</code>	Print the data pack to stdout, return 0 (no LLM call, no files)
<code>--template</code>	<code>render_markdown(record)</code>
<i>(default)</i>	<code>render_markdown_llm(record)</code>
<code>--stdout</code>	Print the report, write no report file
<code>-o PATH</code>	Write to <code>PATH</code> (default <code><CVE>_report.md</code> in the current directory)
<code>--json</code>	Additionally write <code><CVE>_data.json</code> in the current directory — even with <code>--stdout</code>

Exit codes: `0` success · `1` fatal error (`SystemExit` with a message: CVE record not found, LLM problem) · `2` bad command line (argparse, including an invalid CVE ID).

Part 5 — Advanced topics

5.1 Design principles (and why)

Principle	How the code shows it	Why it matters
Standard library only	No <code>requests</code> , no <code>pip install</code>	Runs on any server with Python; one file is easy to audit and copy
Authoritative first, fallbacks second	<code>cvelistV5</code> → <code>cveawg</code> ; local git → <code>git.kernel.org</code> ; CNA → NVD → ADP for CWE	Best data wins; the tool survives outages
Validate what you fetched	<code>fetch_cvelist</code> checks the returned <code>cveId</code>	A CDN can hand back the wrong blob
Fail soft, except where failure means "no report"	Fetchers return <code>None</code> ; only the CVE record and the LLM are fatal	One flaky news site must not block a security report
Never guess official classifications	CWE: "No official record" rather than keyword inference; function names flagged "derived — approximate"	A wrong CWE looks authoritative; a visible gap does not
Keep sources separate, label them	Multiple CVSS entries each tagged with their source	Readers see disagreements instead of a silently merged number
Say what was skipped, and why	The <code>- not included:</code> console list; the Provenance section	Absence of a section is explainable
Separate data from prose	Collect → JSON → render	Data is testable and auditable; renderers are swappable
Ground the LLM	System prompt restricts it to the data pack	Reduces (does not eliminate) invention
Deterministic where possible	Template renderer; temperature 0.2	Reproducibility for the parts that can be reproducible
No silent fallback from LLM to template	<code>llm_chat</code> raises <code>SystemExit</code>	Prevents a user believing they got an LLM-written report when they did not

5.2 The data pack schema

Top-level keys of the record returned by `build_record` (also what `--json` / `--json-only` emit):

Key	Type	Meaning
<code>cve_id</code> , <code>title</code> , <code>description</code>	string	Identity and the upstream commit message
<code>assigner</code> , <code>is_kernel</code>	string, bool	CNA short name; <code>true</code> when it is <code>Linux</code>
<code>vendor</code> , <code>product</code> , <code>state</code>	string	From the CNA's <code>affected</code> block and metadata (<code>PUBLISHED</code> , ...)
<code>published</code> , <code>updated</code> , <code>generated_at</code>	string	ISO dates from the record; UTC time of this run
<code>generator</code>	string	The tool that built the CVE record (e.g. <code>bippy-1.2.0</code>)
<code>subsystem</code>	string	Derived from the title or first file
<code>files</code> , <code>functions</code> , <code>functions_derived</code>	list, list, bool	Affected files; changed functions; whether they were <i>derived</i> from the diff
<code>modules</code> , <code>config</code> , <code>makefiles</code>	lists	Makefile-heuristic results
<code>repos</code>	list	Source repo(s) from the record

Key	Type	Meaning
<code>module_by_branch</code>	object	<code>{branch: [{config, files: [{file, config_symbol, result}]}]}</code>
<code>module_stats</code>	object	<code>{module: {module, rows: [{region, total_ips, loaded, zservices, vms, phys_srv, kvm_host, cont_host, containers}]}]}</code>
<code>module_not_loaded</code> , <code>module_builtin</code>	lists	See 4.9
<code>cvss</code>	list	<code>[{version, vector, score, severity, source}]</code>
<code>cwe</code>	object	<code>{id, ids[], name, source, derived}</code>
<code>epss</code> , <code>kev</code> , <code>ssvc</code>	object / null	Raw EPSS row, the KEV entry (or <code>null</code>), parsed SSVC
<code>versions</code>	object	<code>source, intro_version, intro_commits[], intro_backported[], mainline_version, mainline_commit, fixes[], unfixed[] (+ affected_ranges[] in fallback mode)</code>
<code>cpe_ranges</code>	list	Vulnerable version ranges
<code>references</code>	object	<code>{commits, cve_records, discussion, advisories, other}</code> — lists of URLs
<code>patches</code>	object	<code>{commit_hash: {commit, web_url, author, date, subject, body, files[], functions[], insertions, deletions}}</code>
<code>sources_used</code>	list	Human-readable list of sources that produced data
<code>nvd_status</code>	string	<code>vulnStatus</code> , or <code>Not in NVD</code>
<code>exploitdb</code> , <code>debian</code> , <code>lkc</code> , <code>tuxcare</code>	list / object / null	Vendor and community results
<code>ubuntu_link</code>	string / null	Link only; <code>null</code> under <code>--no-vendor</code> (the template uses it as the "vendor skipped" flag)
<code>feedly</code> , <code>hackerwire</code> , <code>hackernews_coverage</code> , <code>cybersecuritynews_coverage</code> , <code>securityonline_coverage</code>	object / list / null	Media results (<code>null</code> = unreachable or not found; <code>[]</code> = searched, nothing found)
<code>qualys_blog_link</code> , <code>bleepingcomputer_search_link</code>	string / null	Link-only sources; <code>null</code> under <code>--no-media</code>

A real `versions` block (abbreviated), from the repo's sample data:

```
{
  "source": "kernel vulns.git dyad",
  "intro_version": "2.6.12",
  "intro_commits": ["1da177e4c3f41524e886b7f1b8a0c1fc7321cac2"],
  "intro_backported": [],
  "mainline_version": "7.3-rc1",
  "mainline_commit": "7bad4bda74dc4713f398d3b7624ff05478e3a568",
  "fixes": [
    { "commit": "7bad4bda...", "release": "7.3-rc1", "branch": "mainline", "is_mainline": true,
      "intro_version": "2.6.12", "intro_commit": "1da177e4..." },
    { "commit": "46640c81...", "release": "7.2.3", "branch": "7.2.y", "is_mainline": false, "...": "..." }
  ],
  "unfixed": []
}
```

5.3 Error handling model

Situation	What the code does
Invalid CVE ID	<code>ap.error(...)</code> → exit 2
CVE record unreachable / absent	<code>SystemExit("ERROR: could not fetch a record for ...")</code>
Any other HTTP/network failure	<code>_http</code> returns <code>None</code> ; warning on stderr (except 404); run continues
JSON that does not parse	The <code>_http_json</code> / fetcher returns <code>None</code>
Unexpected exception inside a <i>parser of third-party data</i> (TuxCare JSON, cache read)	Swallowed (<code>except Exception: pass</code>) → treated as "no data"
Cache folder unusable	<code>_cached</code> falls back to calling the downloader directly
<code>git show</code> fails or times out (30 s)	Local patch path returns <code>None</code> → remote path
<code>find_module_new</code> import fails	<code>resolve_modules_by_branch</code> returns <code>{}</code>
Worktree creation fails	That branch is skipped
<code>module_stats.py</code> fails, times out (60 s) or prints no table	<code>fetch_module_stats</code> returns <code>None</code> → recorded under <code>module_not_loaded</code>
LLM proxy: unreachable / HTTP error / empty reply	<code>SystemExit</code> with the server's message (first 500 chars)

5.4 Performance and network cost

Everything runs **sequentially in one thread**, so wall time $\approx$ the sum of all request latencies plus git work.

Group	Requests	Notes
Core	CVE record 1 (+1 fallback), dyad 1, NVD 1, EPSS 1, KEV 0–1	KEV cached 6 h
Vendor	Exploit-DB 1, TuxCare 1, Debian 0–1 (cached 24 h), linuxkernelcves 0–1 (cached 24 h)	First run downloads ~12 MB + ~6 MB
Media	Feedly 1, Hacker Wire 1, Hacker News 1–2, CyberSecurityNews 1–2, SecurityOnline 1–2	2nd request only if the 1st search found nothing and a colloquial name exists
Patches	0 network requests when commits are in the local clone; else 1 per commit	A CVE with 8 commits means 8 patches
Makefiles	1–2 per compiled affected file	Always on for kernel CVEs with files
Module resolution	no network; heavy local disk work	Worktree checkouts
LLM	1 POST, up to 180 s	

A test run with `--template --no-module-resolve --no-module-stats` took **about 27 s** on the development machine.

Speed tips: `--no-media` (cuts ~6 requests), `--no-vendor`, `--no-diff`, `--no-module-resolve`; set `NVD_API_KEY`; keep the cache warm when running many CVEs.

5.5 Security and privacy considerations

Topic	Facts	Advice
Command execution	<code>subprocess.run</code> always receives an argument <code>list</code> ; no <code>shell=True</code> . Commit IDs are validated against <code>^[0-9a-f]{8,40}\$</code> before <code>git show</code> . Module names passed to <code>module_stats.py</code> come from a <code>[A-Za-z0-9_-]+</code> regex.	Keep it that way when editing
Parsing untrusted web content	HTML is handled with regex and <code>json.loads</code> only; nothing is executed or evaluated	Safe by construction; but regex parsers can silently mis-parse if a site changes layout
What leaves your machine	(1) Each GET reveals your IP and the CVE ID you are researching to ~15 third parties. (2) In default mode the whole data pack is POSTed to the zLLM proxy, which forwards to whatever upstream model it is configured for (<code>gpt-5.4</code> by default). The pack includes fleet statistics (per-region IP counts, ZService counts) when <code>module_stats.py</code> ran.	Confirm where your proxy sends data and whether that is acceptable for internal inventory numbers. If not, use <code>--template</code> or <code>--no-module-stats</code>
Secrets	<code>NVD_API_KEY</code> and <code>ZLLM_API_KEY</code> come from the environment; nothing is written to disk	Do not hard-code keys
Executed file	<code>--module-stats-script</code> is <code>run</code> . It is only checked with <code>os.path.isfile</code>	Point it only at a script you trust
Temp files	Cache and worktrees live under <code>\$TMPDIR</code>	Use a private <code>TMPDIR</code> on shared hosts

5.6 Extending the tool

Recipe: add a new data source

1. **Fetcher** — one function that returns `None` / `[]` / data (follow the return convention in 4.5):

```
def fetch_example(cve_id):
    """Example.org: per-CVE JSON. 404 is normal for CVEs they do not track."""
    data = _http_json(EXAMPLE_API % cve_id, timeout=15)
    return data # None when unreachable or not found
```

2. **Summariser (optional)** — trim the response to the few fields you need, like `summarize_debian`.
3. **Call it in** `build_record`, behind the right flag (`fetch_vendor` / `fetch_media`) and `is_kernel` check if relevant.
4. **Store it** in the returned dict under a new key.
5. **Declare it** — append to `sources_used` when it produced data, and add a `_skip(name, reason)` line when it did not.
6. **Render it** — add a block in `render_markdown` (template) *and* mention it in `ZLLM_REPORT_USER_TEMPLATE` (LLM), otherwise the model will not know to include it.
7. **Mind the size** — if the data can be large, trim it in `_slim_data_pack`.

The three "unused" sources (Red Hat, Arch, OSV) already have steps 1–2 done; enabling one means steps 3–7.

Other common changes

Goal	Change
Resolve modules for another kernel branch	Add it to <code>MODULE_RESOLVE_BRANCHES</code> ; create a local branch of that name in the Linux clone; add <code>configs/config-<major.minor>.<patch></code> files
Use a different LLM	Point <code>ZLLM_BASE_URL</code> / <code>ZLLM_MODEL</code> at any server that implements OpenAI-style <code>POST /chat/completions</code>
Change report wording or sections	Edit <code>ZLLM_REPORT_SYSTEM_PROMPT</code> / <code>ZLLM_REPORT_USER_TEMPLATE</code> (LLM) or <code>render_markdown</code> (template)
Cache another large download	Wrap its <code>_http</code> call in <code>_cached(name, ttl, producer)</code>
Lift the 350 000-character limit	Change <code>ZLLM_MAX_DATA_PACK_CHARS</code> (only if the proxy allows it)

5.7 Troubleshooting

Symptom	Likely cause	Fix
<code>ERROR: could not reach zLLM proxy</code> ... Is the proxy running?	Proxy not started, or wrong <code>ZLLM_BASE_URL</code>	<code>zllm start</code> , or use <code>--template</code>
<code>ERROR: zLLM request failed (4xx ...)</code>	Request rejected, often content too large	Re-run with <code>--no-media</code> ; check <code>ZLLM_MAX_DATA_PACK_CHARS</code> vs. the proxy's limit
<code>ERROR: could not fetch a record for CVE-...</code>	Typo, CVE not yet published, or no network	Check the ID on cve.org ; check connectivity
<code>! fetch failed: <url> (HTTP Error 429 ...)</code>	Rate-limited (commonly NVD)	Set <code>NVD_API_KEY</code> ; wait and retry
<code>not included: Per-branch module resolution (local Linux source tree not available)</code>	<code>--linux-src</code> wrong	Pass the correct clone path
... (no matching .config files or <code>find_module_new.py</code> unavailable)	Config files not named <code>config-<major.minor>.<patch></code> , or <code>find_module_new.py</code> not beside the script	Rename files; run the script from its own folder
Commit dates blank	Commit not present in the local clone	<code>git fetch</code> in the clone (stable branches included)
Report says a module is "not loaded" but you expect it is	<code>module_stats.py</code> may have failed or timed out (see 5.8, item 5)	Run <code>module_stats.py <module> -brief</code> by hand
Stale Debian / KEV / community data	Cache TTL	<pre>rm -rf "\${TMPDIR:-/tmp}/cve_report_cache"</pre>
First run very slow	Debian dump (~12 MB gz) and <code>linuxkernelcves</code> (~6 MB) downloading	Wait; later runs use the cache
Leftover <code>find_module_wt_*</code> folders in <code>\$TMPDIR</code>	A run was killed hard	<code>git -C <clone> worktree prune</code> , then delete the folders

5.8 Known quirks and limitations of the current code

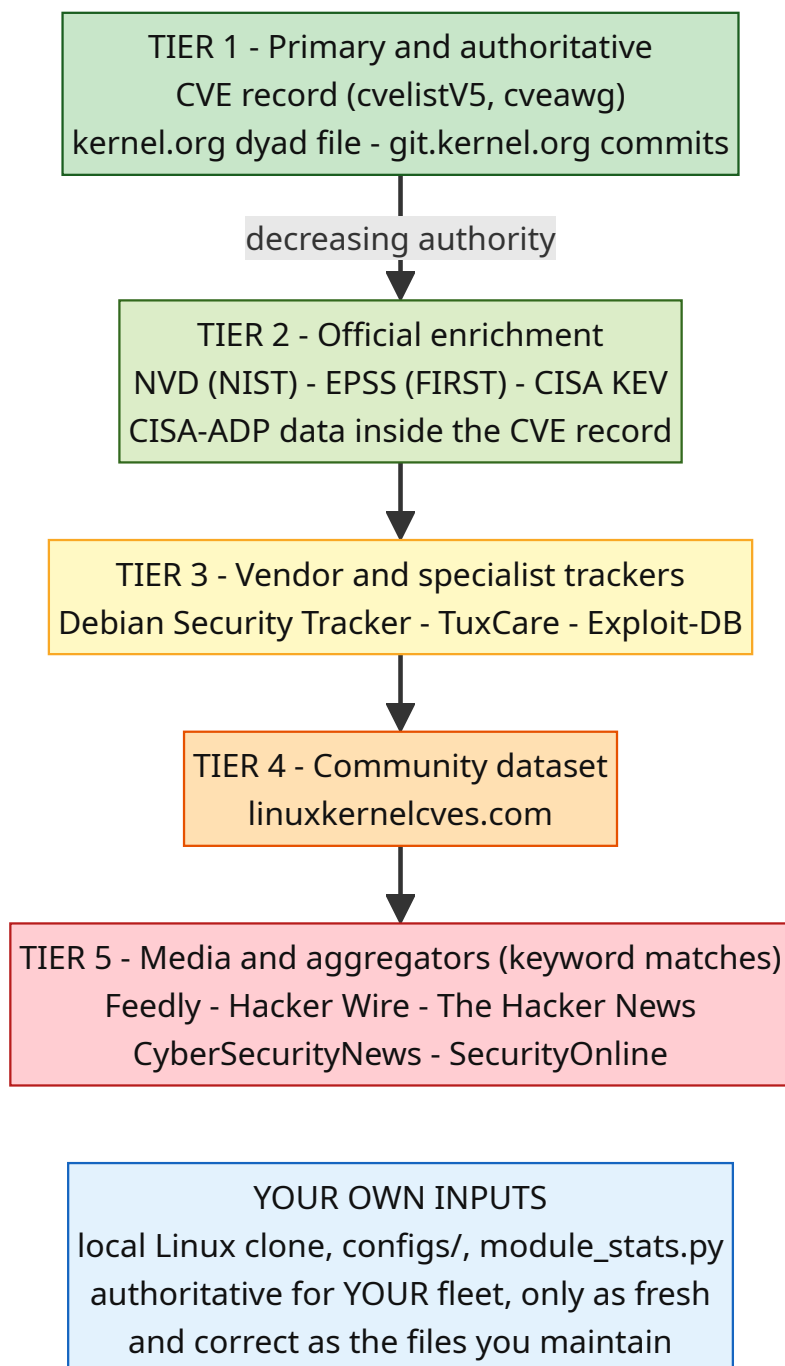
These were found while reading the code against its own comments and README. None of them stops the tool working; they are listed so nobody is surprised.

#	Observation	Effect
1	Unused code: <code>fetch_redhat</code> , <code>fetch_archlinux</code> , <code>fetch_osv</code> , <code>summarize_redhat/archlinux/osv</code> , <code>classify_cwe</code> , <code>CWE_KEYWORDS</code> , the <code>SUSE_CVE_PAGE</code> constant; the <code>title</code> and <code>desc</code> parameters of <code>parse_cwe</code>	Dead weight; also why the file's top docstring lists sources that the report never shows
2	The module docstring lists Red Hat / Arch / OSV / Ubuntu / SUSE as sources, and the <code>--no-vendor</code> help text says it skips "Red Hat/Debian/Arch/OSV/Exploit-DB"	Misleading. In reality <code>--no-vendor</code> skips Exploit-DB, Debian, linuxkernelcves and TuxCare (the README is correct)
3	<code>_func_from_context</code> comments say "last identifier before <code>()</code> " but the code returns the first acceptable one	Function names are hints either way
4	The set of commits to fetch is a Python <code>set</code> , so iteration order can vary between runs	Only matters when the mainline patch yields no function names: <i>which other patch</i> supplies them may differ run to run
5	<code>fetch_module_stats</code> returns <code>None</code> for script missing, crash, timeout, or "module not tracked" , and <code>build_record</code> puts every <code>None</code> in <code>module_not_loaded</code>	A failed or slow stats script can be reported as "not loaded in any region". Verify surprising negatives by hand
6	<code>fetch_makefile_config</code> reads the default branch of the stable tree, not the affected branch	It can disagree with the per-branch table; trust the per-branch table
7	Remote patches carry no date (author date $\neq$ commit date)	Dates are blank for commits absent from the local clone
8	<code>--stdout</code> does not suppress <code>--json</code>	<code><CVE>_data.json</code> is still written
9	If both the raw GitHub file and cveawg return something that does not match the requested ID, the fallback dict is still returned	Very unlikely; the report would describe the wrong record
10	<code>_slim_data_pack</code> does not re-check the size after its last step	An enormous pack could still be rejected by the proxy
11	<code>build_record</code> 's fatal path and <code>llm_chat</code> raise <code>SystemExit</code> rather than a normal exception	Importing the functions into other code needs <code>try/except SystemExit</code>
12	The README calls the script "deterministic"; that is true of the data collection and the <code>--template</code> output, not of LLM prose	Do not diff LLM reports byte-for-byte
13	<code>.claude/commands/cve.md</code> runs <code>python3 /Users/gowtham-23345/CVE_Report/cve_report.py</code> — a macOS-style path	On this Linux machine the repo is at <code>/home/gowtham-23345/CVE_Report</code> ; the <code>/cve</code> command would need that path updated
14	In template mode the introducing commit's date is printed in git's raw format (<code>Sun Aug 1 10:35:52 2010 +0300</code>) while other dates are <code>YYYY-MM-DD</code>	Cosmetic

Part 6 — Official sources and reliability

6.1 The trust model

The tool does not treat all sources equally. In decreasing order of authority:



6.2 Source-by-source assessment

"Official" here means *the body that is the authority for that kind of information*. It does not mean the tool has been endorsed by any of them.

Source	Operated by	Official for...	Interface	Stability of the interface	Main failure mode	How the tool copes
cvelistV5	The CVE Program (the international program that governs CVE IDs; MITRE-operated, CISA-sponsored)	The CVE record itself	Raw JSON file	Stable path scheme	A CDN can return a stale or wrong blob	Validates <code>cveId</code> ; falls back to cveawg
cveawg.mitre.org	The CVE Program	The CVE record (CVE Services API)	REST	Documented service	Outage	Only the fallback
vulns.git dyad	The kernel.org CNA team	Which commits introduce/fix a kernel CVE, per release	Plain text	Stable format (header version <code>1.2.0</code> seen)	File absent for older / non-kernel CVEs	Falls back to the record's ranges; says so in Provenance
git.kernel.org	kernel.org	The kernel source history	Plain/patch endpoints	Stable	Outage, rate limits	Prefers the local clone; retries
NVD 2.0	NIST (US government)	NVD's own CVSS/CWE/CPE analysis and status	REST (documented)	Versioned API	Rate limits; CVE not yet analysed	Optional key; shows status; CVSS also from CNA/ADP
EPSS	FIRST.org	The EPSS probability	REST (documented)	Documented	Outage; score missing for new CVEs	Row simply absent, logged
CISA KEV	CISA (US government)	The known-exploited list	Published JSON feed	Stable	Outage	Cached 6 h; "not listed" is shown explicitly
Debian Security Tracker	The Debian project	Debian's per-suite status	Published JSON dump	Stable	Large download	Cached 24 h
TuxCare tracker	TuxCare (commercial vendor)	TuxCare's own ELS / KernelCare status only	HTML (regex)	Fragile — breaks if the page layout changes	Silent mis-parse → "no entry"	Returns <code>None</code> when no Debian rows are found

Source	Operated by	Official for...	Interface	Stability of the interface	Main failure mode	How the tool copes
Exploit-DB	OffSec	A widely used curated archive of public exploits (not an authority on whether a CVE is exploitable)	Undocumented AJAX endpoint	Fragile — undocumented	Header requirement; layout change	None (unreachable) vs. <code>[]</code> (no hit) kept distinct
linuxkernelcves.com data	Community maintainer	Nothing officially; useful cross-check	JSON file	Depends on the maintainer	No accuracy guarantee (stated in the code)	Shown only as "community cross-check"
Feedly	Feedly (commercial)	Nothing — an aggregator	JSON inside HTML	Fragile	Layout change; noisy chatter	Noisy hosts filtered; capped at 5
Hacker Wire / Hacker News / CyberSecurityNews / SecurityOnline	Independent sites	Nothing — media	<code>ld+json</code> , Blogger feed, WordPress REST	Standard feeds, but search is keyword-based	Irrelevant hits	Only titles and links are shown, capped at 3
Local clone, <code>configs/</code> , <code>module_stats.py</code>	You	Your own kernels and fleet	Files and a script	—	Outdated configs or inventory	Lists exactly which configs were used
Not used: Red Hat, Arch, OSV	Red Hat; Arch Linux; Google's open-source vulnerability project	—	—	—	—	Code exists, never called

Reachability check (documentation time, 2026-10-06). From the machine hosting this repo, each of the endpoints above returned HTTP 200 for `CVE-2026-53359`, except Arch (404 — expected, Arch tracks few CVEs, and the tool does not call it). This was an HTTP-status check only; the content of most sources was exercised by the full run in [Appendix C](#). One request to `raw.githubusercontent.com` got no response within 15 s on the first attempt and succeeded on the next — exactly the situation that `_http`'s retries and `fetch_cvelist`'s fallback exist for. This is a point-in-time observation, not a guarantee.

6.3 What the tool does to protect accuracy

1. **Record validation** — the fetched CVE record's own ID must match the one requested.
2. **Explicit pairing from the kernel team** — versions/commits come from the dyad file, not from guessing across arrays.
3. **No hidden merging** — every CVSS score keeps its source label; the CWE keeps its source; NVD status is shown verbatim.
4. **No invented classification** — no CWE is guessed; derived function lists are labelled approximate.
5. **Cross-check labelling** — the community dataset is explicitly presented as a cross-check with "no accuracy guarantee".
6. **Provenance** — the report lists the sources that actually produced data (and shows "CISA KEV (not listed)" when it queried and found nothing).

7. **Accountability on stderr** — every unused source is listed with a reason.
8. **Grounded prose** — the LLM is instructed to use only supplied facts and to write "Not available" for gaps.

6.4 Verify any claim yourself

Every row of a report can be confirmed at its origin. Replace `CVE-2026-53359` with your ID (and `2026/53xxx` with its year/bucket — see 4.4).

Report field	Confirm at the official place	Command / link
Title, description, dates, state, CNA CVSS, files, functions	The CVE record	<code>curl -s https://cveawg.mitre.org/api/cve/CVE-2026-53359</code> · <code>https://www.cve.org/CVERecord?id=CVE-2026-53359</code>
Introduced / fixed commits and releases	The kernel team's dyad file	<code>curl -s https://git.kernel.org/pub/scm/linux/security/vulns.git/plain/cve/published/2026/CVE-2026-53359.dyad</code>
NVD status, NVD CVSS/CWE	NVD	<code>curl -s "https://services.nvd.nist.gov/rest/json/cves/2.0?cveId=CVE-2026-53359"</code> · <code>https://nvd.nist.gov/vuln/detail/CVE-2026-53359</code>
EPSS	FIRST	<code>curl -s "https://api.first.org/data/v1/epss?cve=CVE-2026-53359"</code>
KEV "listed / not listed"	CISA	<code>curl -s https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json grep -c CVE-2026-53359</code>
Commit author, date, diff	git.kernel.org / your clone	<code>https://git.kernel.org/stable/c/<sha></code> · <code>git -C ~/Linux_Stable/linux show -s --format=%cd <sha></code>
"Is this fix in release X?"	Your clone	<code>git -C ~/Linux_Stable/linux tag --contains <sha> sort -V head</code>
Debian status	Debian	<code>https://security-tracker.debian.org/tracker/CVE-2026-53359</code>
Public exploit	Exploit-DB	<code>https://www.exploit-db.com/search?cve=CVE-2026-53359</code> (in a browser)
Module / <code>CONFIG_</code> result	Your clone and <code>.config</code>	<code>python3 find_module_new.py arch/x86/kvm/mmu/mmu.c ~/Linux_Stable/linux --config configs/config-6.12.90 --branches origin/linux-6.12.y</code> (arguments per <code>--help</code> : files first, then the source tree; it uses a temporary worktree per branch)
Fleet numbers	The fleet script	<code>~/Server-Modules/Scripts/module_stats.py <module> --brief</code>

The report's own **References** section already contains most of these links.

6.5 Reliability of the LLM step — and how to audit it

The LLM is the only non-deterministic, non-verifiable component. The script **does not check the generated text against the data**. The mitigations are in the prompt (grounding, "Not available", temperature 0.2), not in code. To audit a report:

```
# 1. keep the facts alongside the prose
python3 cve_report.py CVE-2026-53359 --json          # writes CVE-2026-53359_report.md and CVE-2026-53359_data.json

# 2. every commit hash in the report must exist in the data pack
grep -oE '\b[0-9a-f]{12,40}\b' CVE-2026-53359_report.md | sort -u | while read h; do
    grep -q "$h" CVE-2026-53359_data.json || echo "NOT IN DATA PACK: $h"
done

# 3. compare against the deterministic rendering
python3 cve_report.py CVE-2026-53359 --template -o /tmp/template.md
diff <(grep -E 'CVSS|EPSS|NVD status' /tmp/template.md) <(grep -E 'CVSS|EPSS|NVD status' CVE-2026-53359_report.md)
```

(Step 2 was run against an existing LLM-written report in this repo and found 20 distinct hashes, all present in its data pack.) Also spot-check scores, dates and fixed-release numbers in section 3, the most error-prone part to re-type.

6.6 What the report is not

- **Not a scan of your systems.** "Fleet exposure" reports *module-load statistics from an inventory*, not whether a specific host is exploitable.
- **Not proof of exploitability or safety.** CVSS, EPSS and SSVC are estimates by third parties.
- **Not a replacement for your patch policy.** Mitigation text is generic (update to a fixed release, migrate off EOL branches) unless the upstream description gives specifics.
- **Not an official publication of any listed organisation.** It is a compilation; the sources are the authority.

Part 7 — Advantages, limits and when to use what

7.1 Advantages

Advantage	Detail
Saves time	One command replaces roughly ten websites and a manual version-matching exercise
Authoritative core	CVE record, kernel.org dyad and git history are primary sources
Kernel-accurate version mapping	Uses the kernel team's explicit pairs; surfaces EOL branches that will never be fixed
Fleet-aware	Links CVE → file → CONFIG_ → module → how many of <i>your</i> servers load it — something no public site can give
Transparent	Provenance section; per-source CVSS and CWE; a list of what was skipped and why
Resilient	Retries, fallbacks, caching, graceful degradation
Zero install	Standard library only; copy one file
Flexible output	Markdown (LLM or template) and JSON for automation
Polite to servers	Identifying User-Agent, caching of large dumps, sequential requests
Extensible	Clear collect → normalise → render split; recipe in 5.6

7.2 Limitations

Limitation	Detail
Kernel-centric	Non-kernel CVEs lose the dyad, patches, modules and fleet sections
Depends on external sites	Layout/API changes can break the fragile fetchers (TuxCare, Exploit-DB, Feedly)
Media section is noisy	Keyword matching, not relevance checking
LLM output is not verified	See 6.5
Per-branch resolution is limited to the configured branches and config files	Others are simply absent
Sequential and I/O-heavy	A full run is slower than a <code>--no-...</code> run; worktree creation dominates
Internal tooling assumptions	Paths and <code>module_stats.py</code> are specific to this environment
Known quirks	See 5.8

7.3 The script vs. the `/cve` command

	<code>python3 cve_report.py ...</code>	<code>/cve ...</code> in Claude Code
What it is	The data engine plus an LLM-proxy or template writer	The same engine, then Claude reads the actual diffs and writes deeper analysis
Strengths	Fast, scriptable, batchable, runs unattended	Root-cause analysis, impact, exploitation discussion, a subsystem primer

	<code>python3 cve_report.py ...</code>	<code>/cve ...</code> in Claude Code
Needs	Python (+ optional extras)	Claude Code session; the command file's script path must be valid (see 5.8, item 13)
Output	<code><CVE>_report.md</code> (+ JSON)	An overwritten <code><CVE>_report.md</code> with extra analysis sections

Appendix A — Command-line reference

```
usage: cve_report.py [-h] [-o OUTPUT] [--stdout] [--json] [--json-only]
                    [--no-diff] [--no-vendor] [--no-media] [--template]
                    [--llm-model LLM_MODEL] [--linux-src LINUX_SRC]
                    [--configs-dir CONFIGS_DIR] [--no-module-resolve]
                    [--module-stats-script MODULE_STATS_SCRIPT]
                    [--no-module-stats]
                    cve
```

Argument	Default	Effect	Sources affected
cve	required	CVE ID such as CVE-2026-53359 (case-insensitive)	—
-o, --output	<CVE>_report.md	Report path	—
--stdout	off	Print the report, write no report file	—
--json	off	Also write <CVE>_data.json	—
--json-only	off	Print the data pack and exit	—
--no-diff	off	Do not fetch commit patches	Patches (dates, authors, derived functions)
--no-vendor	off	Skip vendor/community lookups	Exploit-DB, Debian, linuxkernelcves, TuxCare, ubuntu_link
--no-media	off	Skip media lookups	Feedly, Hacker Wire, Hacker News, CyberSecurityNews, SecurityOnline, Qualys/BleepingComputer links
--template	off	Use the built-in renderer	The LLM
--llm-model	\$ZLLM_MODEL or gpt-5.4	Model id for the proxy	—
--linux-src	~/Linux_Stable/linux	Local Linux git clone	Local patches; module resolution
--configs-dir	./configs	Folder of config-* files	Module resolution
--no-module-resolve	off	Skip per-branch module table	find_module_new.py
--module-stats-script	~/Server-Modules/Scripts/module_stats.py	Fleet script path	Fleet exposure
--no-module-stats	off	Skip fleet exposure	module_stats.py

Files the tool reads or writes

Path	When	Purpose
./<CVE>_report.md (or -o)	Normal run	The report
./<CVE>_data.json	--json	The data pack
\$TMPDIR/cve_report_cache/{kev,debian_tracker,linux_kernel_cves}.json	Vendor/KEV lookups	Download cache

Path	When	Purpose
<code>\$TMPDIR/find_module_wt_<random></code>	Module resolution	Temporary git worktree, removed afterwards

Examples

```
python3 cve_report.py CVE-2026-53359 # LLM-written report
python3 cve_report.py CVE-2026-53359 --template --stdout # print template report
python3 cve_report.py CVE-2026-53359 --json-only > pack.json # facts only
python3 cve_report.py CVE-2026-53359 --no-media --no-module-resolve # fast run
for c in CVE-2024-50264 CVE-2024-53104; do python3 cve_report.py "$c"; done # batch
```

Appendix B — Function index

Line numbers refer to commit `7e38fe1`.

Function	Purpose
<code>_http</code> ^{L234}	GET with retries, gzip handling; returns bytes or <code>None</code>
<code>_http_json</code> ^{L264} / <code>_http_text</code> ^{L274}	Decode <code>_http</code> output
<code>llm_chat</code> ^{L279}	POST to the zLLM proxy; raises <code>SystemExit</code> on any failure
<code>_cached</code> ^{L320}	File cache with TTL
<code>_cve_parts</code> ^{L340}	Split a CVE ID into year / number / bucket
<code>cvelist_url</code> ^{L349} , <code>cvelist_blob_url</code> ^{L357}	Raw-file and web-page URLs for the CVE record
<code>fetch_cvelist</code> ^{L369}	CVE record with validation and fallback
<code>fetch_dyad</code> ^{L391}	Kernel dyad text
<code>fetch_nvd</code> ^{L400} , <code>fetch_epss</code> ^{L410} , <code>fetch_kev</code> ^{L418}	NVD, EPSS, CISA KEV
<code>fetch_redhat</code> ^{L434} , <code>fetch_archlinux</code> ^{L441} , <code>fetch_osv</code> ^{L457}	Unused
<code>fetch_linuxkernelcves</code> ^{L464}	Community dataset entry
<code>fetch_debian_entry</code> ^{L480}	Entry from the cached Debian dump
<code>fetch_exploitdb</code> ^{L501}	Exploit-DB rows (<code>None</code> vs <code>[]</code>)
<code>fetch_wp_coverage</code> ^{L513}	WordPress REST search hits
<code>fetch_hackernews_coverage</code> ^{L536}	Blogger feed search hits
<code>fetch_tuxcare</code> ^{L559}	TuxCare Debian ELS and KernelCare fixes
<code>fetch_hackerwire</code> ^{L594} , <code>fetch_feedly</code> ^{L621}	Embedded-JSON media sources
<code>_unfold_headers</code> ^{L661}	Re-join folded e-mail header lines
<code>_fetch_patch_local</code> ^{L673}	Patch details via <code>git show</code>
<code>fetch_patch</code> ^{L758}	Local-then-remote patch parser
<code>_func_from_context</code> ^{L822}	Guess a function name from a hunk header
<code>fetch_makefile_config</code> ^{L835} , <code>_makefile_for</code> ^{L852}	Makefile-heuristic module and <code>CONFIG_</code>
<code>_module_names_from_resolution</code> ^{L892}	Collect <code>.ko</code> names from resolution results
<code>fetch_module_stats</code> ^{L910}	Run and parse <code>module_stats.py</code>
<code>resolve_modules_by_branch</code> ^{L944}	Per-branch, per-config module resolution
<code>_dig</code> ^{L1017}	Safe nested dict lookup
<code>_sev_from_score</code> ^{L1027} , <code>_branch_of</code> ^{L1043}	Score → label; release → branch
<code>classify_cwe</code> ^{L1051}	Keyword CWE guesser — unused
<code>parse_dyad</code> ^{L1064} , <code>_ver_key</code> ^{L1086} , <code>versions_from_dyad</code> ^{L1091} , <code>versions_from_record</code> ^{L1135}	Version/fix mapping

Function	Purpose
<code>parse_cvss</code> ^{L1158} , <code>parse_cwe</code> ^{L1200} , <code>parse_ssvc</code> ^{L1249} , <code>parse_cpe_ranges</code> ^{L1341}	Record parsers
<code>summarize_redhat</code> ^{L1267} , <code>summarize_debian</code> ^{L1287} , <code>summarize_archlinux</code> ^{L1301} , <code>summarize_osv</code> ^{L1312} , <code>summarize_exploitdb</code> ^{L1324}	Reduce source data to report fields (Red Hat/Arch/OSV unused)
<code>categorize_refs</code> ^{L1368} , <code>derive_subsystem</code> ^{L1393} , <code>_title_from_desc</code> ^{L1409}	Reference sorting, subsystem and title heuristics
<code>build_record</code> ^{L1420}	The orchestrator — returns the data pack
<code>_fmt_date</code> ^{L1743} , <code>_fmt_date_hdr</code> ^{L2249} , <code>_short</code> ^{L1753}	Formatting helpers
<code>_slim_patches</code> ^{L1763} , <code>_slim_data_pack</code> ^{L1790}	Shrink the pack for the LLM
<code>render_markdown_llm</code> ^{L1838} , <code>render_markdown</code> ^{L1858}	The two renderers
<code>main</code> ^{L2266}	CLI entry point

Appendix C — A real run, annotated

Command (run on 2026-10-06):

```
python3 cve_report.py CVE-2026-53359 --template --no-module-resolve --no-module-stats --stdout
```

Console (stderr), line by line

Line	What is happening
[*] CVE-2026-53359: fetching cvelistV5 record ...	fetch_cvelist — the one mandatory download
[*] fetching dyad / NVD / EPSS / KEV ...	Core facts: version pairs, scores, exploit probability, known-exploited status
[*] fetching Exploit-DB ...	Vendor group begins
[*] fetching Debian tracker / linuxkernelcves.com (cached, first run may be slow) ...	The two large cached dumps
[*] fetching TuxCare tracker ...	HTML source
[*] fetching media/community coverage (...) ...	The five media sources
[*] fetching 8 commit patch(es) ...	1 introducing commit + 7 fix commits (6 stable + mainline) — read from the local clone
[*] deriving module / CONFIG ...	fetch_makefile_config against git.kernel.org
- not included: Per-branch module resolution (disabled (-no-module-resolve))	Skipped by flag; the reason is printed
- not included: Red Hat ... / Arch ... / OSV.dev (not queried (excluded from reports by design))	The three deliberately unused sources
- not included: linuxkernelcves.com (CVE not present in community dataset)	Queried, no entry — not an error

Report excerpt → where each row came from

Row	Value in the run	Source
Assigner (CNA)	Linux	CVE record → sets is_kernel
Published / Last updated	2026-07-04 / 2026-10-03	CVE record metadata
CVSS	8.8 HIGH (v3.1), AV:L/AC:L/PR:L/UI:N/S:C/C:H/I:H/A:H, CNA (Linux)	CNA container metrics
Weakness	CWE-416 (NVD)	CNA gave none, so NVD was used (see 4.7)
EPSS	0.17 % (6.2th percentile)	FIRST EPSS
CISA KEV	not listed	KEV catalogue searched, no match
CISA SSVC	Exploitation= poc , Automatable=no, Tech-impact=total	CISA-ADP block in the CVE record
NVD status	Modified	NVD vulnStatus
Public exploit	not listed on Exploit-DB	Exploit-DB returned []
Subsystem	KVM: x86	Derived from the title

Row	Value in the run	Source
Module / config	kvm.ko, CONFIG_KVM_X86	Makefile heuristic (arch/x86/kvm/Makefile)
Function	kvm_mmu_get_child_sp() (derived — approximate)	Hunk header of the mainline patch
Introduced	2.6.36, commit 2032a93d66fa	dyad file
Fixed per branch	5.15.222, 6.1.177, 6.6.144, 6.12.95, 6.18.38, 7.1.3, mainline 7.2	dyad file; dates from the local clone
Debian	bookworm 6.1.177-1, trixie 6.12.95-1, sid / forky 7.1.3-1	Debian Security Tracker
TuxCare	Debian12 KCARE FIX (Released), Debian13 (Planned)	TuxCare HTML table
CVE record generated by	bippy-1.2.0	x_generator.engine in the record

Appendix D — FAQ

Does it work for non-kernel CVEs? Partly. You still get the CVE record, scores, EPSS, KEV, NVD, Exploit-DB and media, but no dyad file, patches, module resolution, Debian/TuxCare/linuxkernelcves or fleet exposure.

Why does the report say "Not available" or "Not yet assessed"? The data was missing (for instance NVD has not analysed the CVE yet). The tool and the LLM are told not to guess.

Why are there several CVSS scores? The CNA, CISA-ADP and NVD can each publish a score. The tool shows them separately, with sources, so disagreements are visible.

A file says "built into vmlinux". Is the fleet safe because no module is loaded? No — the opposite. Built-in code is part of every kernel built with that `CONFIG_` symbol, so "module not loaded" does not apply.

How fresh is the data? Live on each run, except CISA KEV (cached 6 h) and the Debian and linuxkernelcves dumps (cached 24 h).

Can I run it for many CVEs? Yes (see the batch loop in Appendix A). The big dumps are shared through the cache. Set `NVD_API_KEY` to avoid NVD rate limits.

Can I use a different language model? Yes: set `ZLLM_BASE_URL` / `ZLLM_MODEL` (or `--llm-model`) to any server that offers an OpenAI-style `/chat/completions` endpoint.

Is the report an official document? No. It is a compilation of official and unofficial sources; Part 6 shows how to confirm any statement at its origin.

Where do I add Red Hat or OSV back? The fetch/summarise functions exist already; follow the recipe in 5.6 (steps 3–7).

Why does the script exit instead of falling back when the LLM proxy is down? To avoid giving you a different style of report without telling you. Add `--template` yourself when you want that.